



Credit Card approval project

Introduction

A credit card is a physical payment card issued by a financial institution, typically a bank, that allows you to borrow money within a pre-approved credit limit. You can use this credit to purchase goods and services or withdraw cash from ATMs.

Here are some key things to know about credit cards:

Functionality:

1. **Borrowing money:** You are essentially borrowing money from the issuer up to your credit limit. This means you need to pay back the borrowed amount, usually within a monthly billing cycle.
2. **Interest charges:** If you don't pay back the balance in full by the due date, you will be charged interest on the remaining amount. The interest rate can vary depending on the card and your creditworthiness.
3. **Repayment:** You can typically make payments towards your credit card balance through online banking, mobile apps, or by mail.

Benefits:

1. **Convenience:** Credit cards offer a convenient way to pay for things without carrying cash. They are also widely accepted by merchants, both online and in-person.
2. **Rewards:** Many credit cards offer rewards programs that give you points, miles, or cash back for your purchases. These can be redeemed for various rewards, such as travel, merchandise, or gift cards.
3. **Building credit:** Using a credit card responsibly and paying your bills on time can help you build a good credit history. This can make it easier to qualify for loans, mortgages, and other forms of credit in the future.

Drawbacks:

1. **Debt:** If you are not careful, it's easy to overspend with a credit card and accumulate debt. This can lead to high interest charges and financial difficulties.
2. **Annual fees:** Some credit cards, especially those with generous rewards programs, may have annual fees.
3. **Interest rates:** Credit card interest rates can be high, so it's important to pay your balance in full every month to avoid paying interest.

Importance of this project

Credit card approval projects can be important for several reasons, both for customers and for financial institutions. Here are some key aspects to consider:

1. For financial institutions:
 - a. **Reduced Risk:** Accurate approval models enable lending to creditworthy customers, minimizing defaults and losses.
 - b. **Increased Profitability:** Approving more eligible customers expands the customer base, leading to higher transaction volume and revenue.
 - c. **Improved Efficiency:** Automated models streamline the application process, saving time and resources.
 - d. **Data-driven Decisions:** Models provide insights into customer behavior and risk factors, facilitating informed credit decisions.
 - e. **Competitive Advantage:** Advanced models offer institutions a competitive edge in attracting and retaining customers.
2. For customers:
 - a. **Faster Access to Credit:** Accurate models offer quicker approval decisions, reducing waiting times and frustration.
 - b. **Fairer Lending:** Models can mitigate bias and ensure credit access is based on objective criteria.
 - c. **Improved Financial Inclusion:** Expanding access to credit can promote financial health and well-being.
 - d. **Personalized Offers:** Models can help tailor credit card options to individual needs and financial goals.
 - e. **Increased Financial Security:** Responsible credit card use can build credit history, providing access to better terms and rates in the future.

Impact on Bank sector

Banks receive a lot of credit card applications. Many of the applications do not get approved for a variety of reasons, like increased loan balances or poor-income levels. Manually analysing these applications can be very time consuming and full

of human errors. Hence we can automate this task with the help of machine learning.

Objective

The objective of this project is to build a machine-learning model that accurately predicts credit card approval outcomes. The model can provide insights into the most important factors that impact credit card approval, allowing banks to make informed decisions quickly and accurately. This can result in a faster and more efficient credit card approval process, reducing the time and cost associated with manual credit assessment methods.

Overall, the proposed project has the potential to upgrade the credit card approval process, reduce manual effort and errors, and enhance the customer experience. This project demonstrates the potential of machine learning in automating and improving critical banking processes and can have significant benefits for banks and financial institutions in the global market.

Steps for the project:

1. Import libraries and dataset
 - Import libraries like pandas, numpy, sklearn, matplotlib, seaborn. Next we will import the two datasets in the form of csv files using pandas dataframes.
 - Next we will merge the two dataframes into one dataframe for data preprocessing
2. Data Preprocessing
 - We will perform data preprocessing techniques like handling missing values, removing duplicates, handling outliers, data scaling and normalization, encoding categorical data, feature scaling and handling imbalanced data.

1. Import the libraries and datasets

```
In [2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
import warnings
warnings.filterwarnings('ignore')
```

C:\Users\Abhishek\AppData\Local\Temp\ipykernel_24820\2591034107.py:1: DeprecationWarning:
Pyarrow will become a required dependency of pandas in the next major release of pandas (pandas 3.0),
(to allow more performant data types, such as the Arrow string type, and better interoperability with other libraries)
but was not found to be installed on your system.
If this would cause problems for you,
please provide us feedback at <https://github.com/pandas-dev/pandas/issues/54466>

```
import pandas as pd
```

```
In [3]: df1=pd.read_csv("C:/Users/Abhishek/Desktop/OdinSchool Program/Capstone project  
df1
```

```
Out[3]:
```

	Ind_ID	GENDER	Car_Owner	Propert_Owner	CHILDREN	Annual_income
0	5008827	M	Y	Y	0	180000.0
1	5009744	F	Y	N	0	315000.0
2	5009746	F	Y	N	0	315000.0
3	5009749	F	Y	N	0	NaN
4	5009752	F	Y	N	0	315000.0
...	...	...	...	...	...	...
1543	5028645	F	N	Y	0	NaN
1544	5023655	F	N	N	0	225000.0
1545	5115992	M	Y	Y	2	180000.0
1546	5118219	M	Y	N	0	270000.0
1547	5053790	F	Y	Y	0	225000.0

1548 rows × 7 columns

```
In [4]: df1.shape
```

Out[4]: (1548, 18)

```
In [5]: df2=pd.read_csv("C:/Users/Abhishek/Desktop/OdinSchool Program/Capstone project  
df2
```

Out[5]:

	Ind_ID	label
--	--------	-------

	0	5008827	1
	1	5009744	1
	2	5009746	1
	3	5009749	1
	4	5009752	1
	...	...	...
	1543	5028645	0
	1544	5023655	0
	1545	5115992	0
	1546	5118219	0
	1547	5053790	0

1548 rows × 2 columns

```
In [6]: df2.shape
```

Out[6]: (1548, 2)

If you notice that 'label' column in df2 can be placed in df1 for easy analysis. We can use the merge function which merges df1 and df2 using the 'Ind_ID' column. Lets label the new dataframe 'df'.

```
In [7]: df=pd.merge(df1,df2,how='outer',on='Ind_ID') #how is the type of join mentioned  
df
```

Out[7]:

	Ind_ID	GENDER	Car_Owner	Propert_Owner	CHILDREN	Annual_income
0	5008827	M	Y	Y	0	180000.0
1	5008865	F	Y	Y	2	135000.0
2	5008889	F	N	Y	0	247500.0
3	5009000	M	Y	Y	0	157500.0
4	5009023	F	N	Y	2	216000.0
...	...	...	...	...	...	...
1543	5150038	M	N	N	0	180000.0
1544	5150047	F	N	Y	0	283500.0
1545	5150164	F	N	N	0	135000.0
1546	5150221	M	Y	Y	0	116100.0
1547	5150412	F	N	Y	0	67500.0

1548 rows × 19 columns

In [8]: `df.shape`

Out[8]: (1548, 19)

In [9]: `df.info()`

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1548 entries, 0 to 1547
Data columns (total 19 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Ind_ID                1548 non-null   int64
1   GENDER                1541 non-null   object
2   Car_Owner             1548 non-null   object
3   Propert_Owner         1548 non-null   object
4   CHILDREN              1548 non-null   int64
5   Annual_income         1525 non-null   float64
6   Type_Income           1548 non-null   object
7   EDUCATION             1548 non-null   object
8   Marital_status        1548 non-null   object
9   Housing_type          1548 non-null   object
10  Birthday_count        1526 non-null   float64
11  Employed_days         1548 non-null   int64
12  Mobile_phone          1548 non-null   int64
13  Work_Phone            1548 non-null   int64
14  Phone                 1548 non-null   int64
15  EMAIL_ID              1548 non-null   int64
16  Type_Occupation       1060 non-null   object
17  Family_Members        1548 non-null   int64
18  label                 1548 non-null   int64
dtypes: float64(2), int64(9), object(8)
memory usage: 229.9+ KB

```

We can see that there are missing values in

'GENDER','Annual_income','Birthday_count','Type_Occupation' columns.

- int - 9 columns-
'Ind_ID','CHILDREN','Employed_days','Mobile_phone','Work_Phone','Phone','
- object - 8 columns-
'GENDER','Car_Owner','Propert_Owner','Type_Income','EDUCATION','Marital_
- float - 2 columns- 'Annual_income','Birthday_count'

2. Data Preprocessing

Check null values - isnull() is widely been in all pre-processing steps to identify null values in the data

```
In [10]: df.isnull().sum()
```

```
Out[10]: Ind_ID          0
        GENDER          7
        Car_Owner       0
        Propert_Owner    0
        CHILDREN         0
        Annual_income    23
        Type_Income      0
        EDUCATION        0
        Marital_status   0
        Housing_type     0
        Birthday_count   22
        Employed_days    0
        Mobile_phone     0
        Work_Phone       0
        Phone            0
        EMAIL_ID         0
        Type_Occupation  488
        Family_Members   0
        label           0
        dtype: int64
```

```
In [11]: # calculate the percentage of missing values in each column
miss_percent = ((df.isnull().sum()/len(df))*100).round(decimals=2)
miss_percent
```

```
Out[11]: Ind_ID          0.00
        GENDER          0.45
        Car_Owner       0.00
        Propert_Owner    0.00
        CHILDREN         0.00
        Annual_income    1.49
        Type_Income      0.00
        EDUCATION        0.00
        Marital_status   0.00
        Housing_type     0.00
        Birthday_count   1.42
        Employed_days    0.00
        Mobile_phone     0.00
        Work_Phone       0.00
        Phone            0.00
        EMAIL_ID         0.00
        Type_Occupation  31.52
        Family_Members   0.00
        label           0.00
        dtype: float64
```

```
In [12]: df['GENDER'].unique()
```

```
Out[12]: array(['M', 'F', nan], dtype=object)
```

```
In [13]: df[df['GENDER']=='M'].count()
```



```
Out[13]: Ind_ID          568
        GENDER          568
        Car_Owner       568
        Propert_Owner   568
        CHILDREN        568
        Annual_income   559
        Type_Income     568
        EDUCATION       568
        Marital_status  568
        Housing_type    568
        Birthday_count  558
        Employed_days   568
        Mobile_phone    568
        Work_Phone      568
        Phone           568
        EMAIL_ID        568
        Type_Occupation  438
        Family_Members  568
        label           568
        dtype: int64
```

```
In [14]: df[df['GENDER']=='F'].count()
```

```
Out[14]: Ind_ID          973
        GENDER          973
        Car_Owner       973
        Propert_Owner   973
        CHILDREN        973
        Annual_income   959
        Type_Income     973
        EDUCATION       973
        Marital_status  973
        Housing_type    973
        Birthday_count  961
        Employed_days   973
        Mobile_phone    973
        Work_Phone      973
        Phone           973
        EMAIL_ID        973
        Type_Occupation  617
        Family_Members  973
        label           973
        dtype: int64
```

Assuming that majority of the customers are females, we shall fill the missing values with 'F'

```
In [15]: df['GENDER']=df['GENDER'].fillna('F')
        df['GENDER'].unique()
```

```
Out[15]: array(['M', 'F'], dtype=object)
```

```
In [16]: df['GENDER'].value_counts()
```

```
Out[16]: GENDER
F      980
M      568
Name: count, dtype: int64
```

For annual_income column we shall replace the null values with the mean income

```
In [17]: mean_annual_inocme = df['Annual_income'].mean()
mean_annual_inocme
```

```
Out[17]: 191399.3262295082
```

```
In [18]: df['Annual_income']=df['Annual_income'].fillna(mean_annual_inocme)
```

```
In [19]: df['Birthday_count']=df['Birthday_count'].fillna(df['Birthday_count'].mean())
```

```
In [20]: df['Type_Occupation'].value_counts()
```

```
Out[20]: Type_Occupation
Laborers                268
Core staff              174
Managers               136
Sales staff            122
Drivers                 86
High skill tech staff   65
Medicine staff          50
Accountants             44
Security staff          25
Cleaning staff          22
Cooking staff           21
Private service staff   17
Secretaries              9
Low-skill Laborers       9
Waiters/barmen staff     5
HR staff                 3
Realty agents            2
IT staff                 2
Name: count, dtype: int64
```

```
In [21]: df['Type_Occupation']=df['Type_Occupation'].fillna('Laborers')
```

```
In [22]: df.isnull().sum()
```

```

Out[22]: Ind_ID          0
        GENDER          0
        Car_Owner       0
        Propert_Owner   0
        CHILDREN        0
        Annual_income   0
        Type_Income     0
        EDUCATION       0
        Marital_status  0
        Housing_type    0
        Birthday_count  0
        Employed_days   0
        Mobile_phone    0
        Work_Phone      0
        Phone           0
        EMAIL_ID        0
        Type_Occupation  0
        Family_Members  0
        label           0
        dtype: int64

```

There are no null values. There might be some cases where same person applied more than once. Now we are going to check if there are any duplicate entries in the dataset.

Check duplicate values - `nunique()` based on several unique values in each column and the data description, we can identify the continuous and categorical columns in the data. Duplicated data can be handled or removed based on further analysis

```

In [23]: df.nunique()

```

```

Out[23]: Ind_ID          1548
        GENDER           2
        Car_Owner        2
        Propert_Owner    2
        CHILDREN         6
        Annual_income    116
        Type_Income       4
        EDUCATION        5
        Marital_status    5
        Housing_type      6
        Birthday_count   1271
        Employed_days     956
        Mobile_phone      1
        Work_Phone        2
        Phone             2
        EMAIL_ID          2
        Type_Occupation   18
        Family_Members    7
        label             2
        dtype: int64

```

```
In [24]: df.duplicated().value_counts() # checks duplicate entries
```

```
Out[24]: False      1548  
         Name: count, dtype: int64
```

```
In [25]: # checking for duplicate values without considering the 'Ind_ID' column  
duplicates = df.duplicated(subset=['GENDER', 'Car_Owner', 'Propert_Owner', 'CHILDREN',  
                                   'Marital_status', 'Housing_type', 'Birthday',  
                                   'Phone', 'EMAIL_ID', 'Type_Occupation', 'Family_Income'])  
duplicates.value_counts()
```

```
Out[25]: False      1386  
         True       162  
         Name: count, dtype: int64
```

```
In [26]: df.shape
```

```
Out[26]: (1548, 19)
```

```
In [27]: df = df.drop_duplicates(subset=['GENDER', 'Car_Owner', 'Propert_Owner', 'CHILDREN',  
                                       'Marital_status', 'Housing_type', 'Birthday',  
                                       'Phone', 'EMAIL_ID', 'Type_Occupation', 'Family_Income'])  
df.shape
```

```
Out[27]: (1386, 19)
```

We have successfully removed 162 rows fo duplicate data

```
In [28]: # Lets rename some of the columns accurately  
df = df.rename(columns={"GENDER": "Gender", "CHILDREN": "Children", "EMAIL_ID": "Email_ID",  
                       "Marital_status": "Marital_Status", "Housing_type": "Housing_Type",  
                       "Birthday": "BirthDay", "Phone": "Phone", "Type_Occupation": "Type_Occupation",  
                       "Family_Income": "Annual_Income", "Ind_ID": "Ind_ID", "label": "Label"})  
df.head(3)
```

```
Out[28]:
```

	Ind_ID	Gender	Car_Owner	Property_Owner	Children	Annual_Income	Type_Occupation
0	5008827	M	Y	Y	0	180000.0	
1	5008865	F	Y	Y	2	135000.0	
2	5008889	F	N	Y	0	247500.0	C

```
In [29]: df.shape
```

```
Out[29]: (1386, 19)
```

```
In [30]: df.describe()
```

Out[30]:

	Ind_ID	Children	Annual_Income	Birthday_count	Employed_da
count	1.386000e+03	1386.000000	1.386000e+03	1386.000000	1386.0000
mean	5.078626e+06	0.424242	1.895100e+05	-16034.055213	60265.0735
std	4.170620e+04	0.792813	1.060417e+05	4183.212232	138577.3223
min	5.008827e+06	0.000000	3.375000e+04	-24946.000000	-14887.0000
25%	5.044507e+06	0.000000	1.215000e+05	-19463.750000	-3153.7500
50%	5.069390e+06	0.000000	1.692000e+05	-15755.000000	-1542.5000
75%	5.115609e+06	1.000000	2.250000e+05	-12531.000000	-412.0000
max	5.150412e+06	14.000000	1.575000e+06	-7705.000000	365243.0000

In [31]: `df.to_csv('C:/Users/Abhishek/Desktop/OdinSchool Program/Capstone project/Final`

3. EDA (Univariate and Bivariate Analysis)

We shall now perform univariate and bivariate analysis. Univariate and bivariate analysis are fundamental techniques in data analysis.

1. Univariate Analysis:- Uni means one and variate means variable, so in univariate analysis, there is only one dependable variable.

Objectives:-

- Characterize its distribution: Central tendency (mean, median, mode), spread (variance, standard deviation, range), shape (skewness, kurtosis).
- Identify outliers: Extreme values that deviate significantly from the rest of the data.
- Detect potential errors or anomalies: Data quality issues or unusual patterns.
- Gain insights into individual variables: Their characteristics and potential issues.

Methods:-

Numerical variables:-

- Frequency distribution tables
- Histogram
- Box plot
- Density plot

Categorical variables

- Pie chart
- Count plot
- Bar chart
- Column chart

2. Bivariate Analysis:- Bi means two and variate means variable, so here there are two variables. The analysis is related to cause and the relationship between the two variables.

Objectives:-

- Explore correlations: Strength and direction of linear relationships.
- Identify associations: Potential patterns or connections between variables.
- Generate hypotheses for further investigation: Ideas for more in-depth analysis.
- Uncover potential explanatory factors: Variables that might influence others.

Methods:-

Numerical variables

- Scatter plot
- Linear Correlation
- Pairplot

Categorical variables

- Chi-Square test
- Z-test and T-test
- Analysis of Variance (ANOVA)

```
In [32]: # Lets divide the variables into two groups (Categorical variables and Numerical variables)
cat_cols=df.select_dtypes(include=['object']).columns
num_cols=df.select_dtypes(include=np.number).columns
print('Categorical variables:-')
print(cat_cols)
print('Numerical variables:-')
print(num_cols)
```

Categorical variables:-

```
Index(['Gender', 'Car_Owner', 'Property_Owner', 'Type_Income', 'Education',  
      'Marital_status', 'Housing_type', 'Type_Occupation'],  
      dtype='object')
```

Numerical variables:-

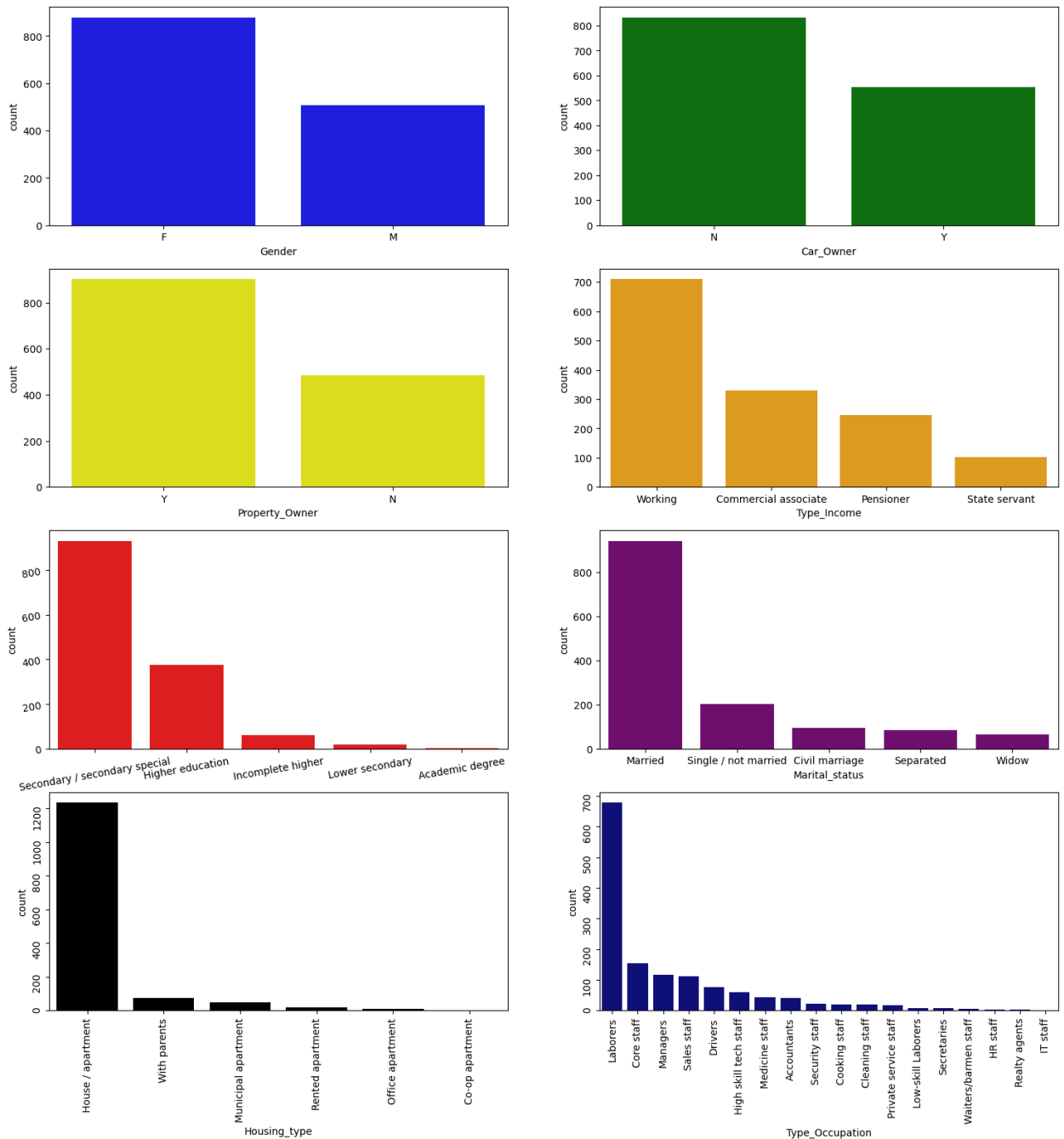
```
Index(['Ind_ID', 'Children', 'Annual_Income', 'Birthday_count',  
      'Employed_days', 'Mobile_phone', 'Work_Phone', 'Phone', 'Email_ID',  
      'Family_Members', 'Label'],  
      dtype='object')
```

EDA (Univariate Analysis)

(a) Countplot

```
In [33]: # Univariate Analysis (Categorical variables) using countplot  
fig, axes = plt.subplots(4, 2, figsize=(18, 18))  
fig.suptitle('Bar plot for all categorical variables in the dataset')  
sns.countplot(ax=axes[0, 0], x='Gender', data=df, color='blue', order=df['Gender'].  
sns.countplot(ax=axes[0, 1], x='Car_Owner', data=df, color='green', order=df['Car_Owner'].  
sns.countplot(ax=axes[1, 0], x='Property_Owner', data=df, color='yellow', order=df['Property_Owner'].  
sns.countplot(ax=axes[1, 1], x='Type_Income', data=df, color='orange', order=df['Type_Income'].  
sns.countplot(ax=axes[2, 0], x='Education', data=df, color='red', order=df['Education'].  
sns.countplot(ax=axes[2, 1], x='Marital_status', data=df, color='purple', order=df['Marital_status'].  
sns.countplot(ax=axes[3, 0], x='Housing_type', data=df, color='black', order=df['Housing_type'].  
sns.countplot(ax=axes[3, 1], x='Type_Occupation', data=df, color='darkblue', order=df['Type_Occupation'].  
  
axes[2][0].tick_params(labelrotation=10);  
axes[3][0].tick_params(labelrotation=90);  
axes[3][1].tick_params(labelrotation=90);
```

Bar plot for all categorical variables in the dataset

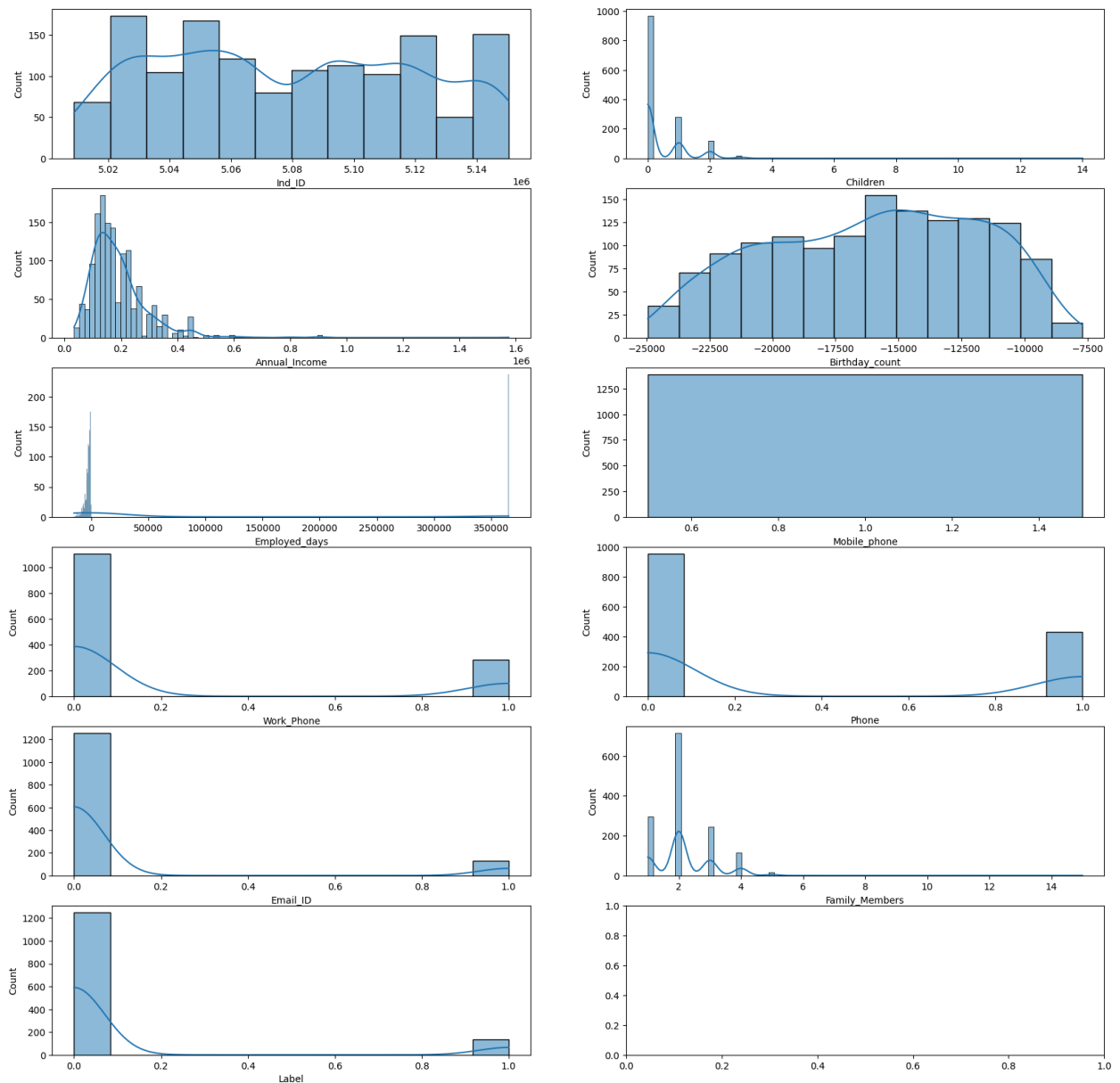


```
In [34]: # Univariate Analysis (Numerical variables) using histplot
fig, axes = plt.subplots(6, 2, figsize=(20, 20))
sns.histplot(ax=axes[0, 0], data=df['Ind_ID'], kde=True)
sns.histplot(ax=axes[0, 1], data=df['Children'], kde=True)
sns.histplot(ax=axes[1, 0], data=df['Annual_Income'], kde=True)
sns.histplot(ax=axes[1, 1], data=df['Birthday_count'], kde=True)
sns.histplot(ax=axes[2, 0], data=df['Employed_days'], kde=True)
sns.histplot(ax=axes[2, 1], data=df['Mobile_phone'], kde=True)
```



```
sns.histplot(ax=axes[3,0],data=df['Work_Phone'],kde=True)
sns.histplot(ax=axes[3,1],data=df['Phone'],kde=True)
sns.histplot(ax=axes[4,0],data=df['Email_ID'],kde=True)
sns.histplot(ax=axes[4,1],data=df['Family_Members'],kde=True)
sns.histplot(ax=axes[5,0],data=df['Label'],kde=True)
print("Skew of Ind_ID:",df['Ind_ID'].skew())
print("Skew of Children:",df['Children'].skew())
print("Skew of Annual_Income:",df['Annual_Income'].skew())
print("Skew of Birthday_count:",df['Birthday_count'].skew())
print("Skew of Mobile_phone:",df['Mobile_phone'].skew())
print("Skew of Work_Phone:",df['Work_Phone'].skew())
print("Skew of Phone:",df['Phone'].skew())
print("Skew of Email_ID:",df['Email_ID'].skew())
print("Skew of Family_Members:",df['Family_Members'].skew())
print("Skew of Label:",df['Label'].skew())
plt.show()
```

```
Skew of Ind_ID: 0.08794402582607747
Skew of Children: 4.75084174853447
Skew of Annual_Income: 3.356535157252575
Skew of Birthday_count: -0.1787408656388059
Skew of Mobile_phone: 0.0
Skew of Work_Phone: 1.463773053869257
Skew of Phone: 0.817638584737677
Skew of Email_ID: 2.7751036728278424
Skew of Family_Members: 2.3323742104930436
Skew of Label: 2.6911239182819355
```



Insights:-

- Majority of the applicants are female which shows that the majority of the customers are women.
- Majority of the applicants do not own a car but own property.
- Majority of the applicants don't have children.
- From Annual Income histogram, the plot is right skewed or positive skewed. Here $\text{mean} > \text{median} > \text{mode}$ are all positive values and not zero or negative. A large no. of data are placed on the left side.
- From the above histogram we can observe most of the applicants source of income is through working.
- Most of the applicants education level is secondary/secondary special.
- Most of the applicants are married.

- Most of the applicants own house/apartments.
- Birthday count values are normally distributed. The plot is almost zero skewed i.e; the plot is symmetrical.
- Many of the applicants have lesser employed days.
- Every applicant has a mobile phone.
- Majority of the applicants don't have a working phone.
- Majority of the applicants don't have Email-ID.
- Most of the applicants are working as labourers.
- Majority of the applicants only have two members in a family
- Most of the applicants were eligible for credit card, however there were applicants whose application were rejected.

EDA (Bivariate Analysis)

When we talk about bivariate analysis, it means analyzing 2 variables. Since we know there are numerical and categorical variables, there is a way of analyzing these variables as shown below:

1. Numerical vs. Numerical

- Scatterplot
- Line plot
- Heatmap for correlation
- Joint plot

2. Categorical vs. Numerical

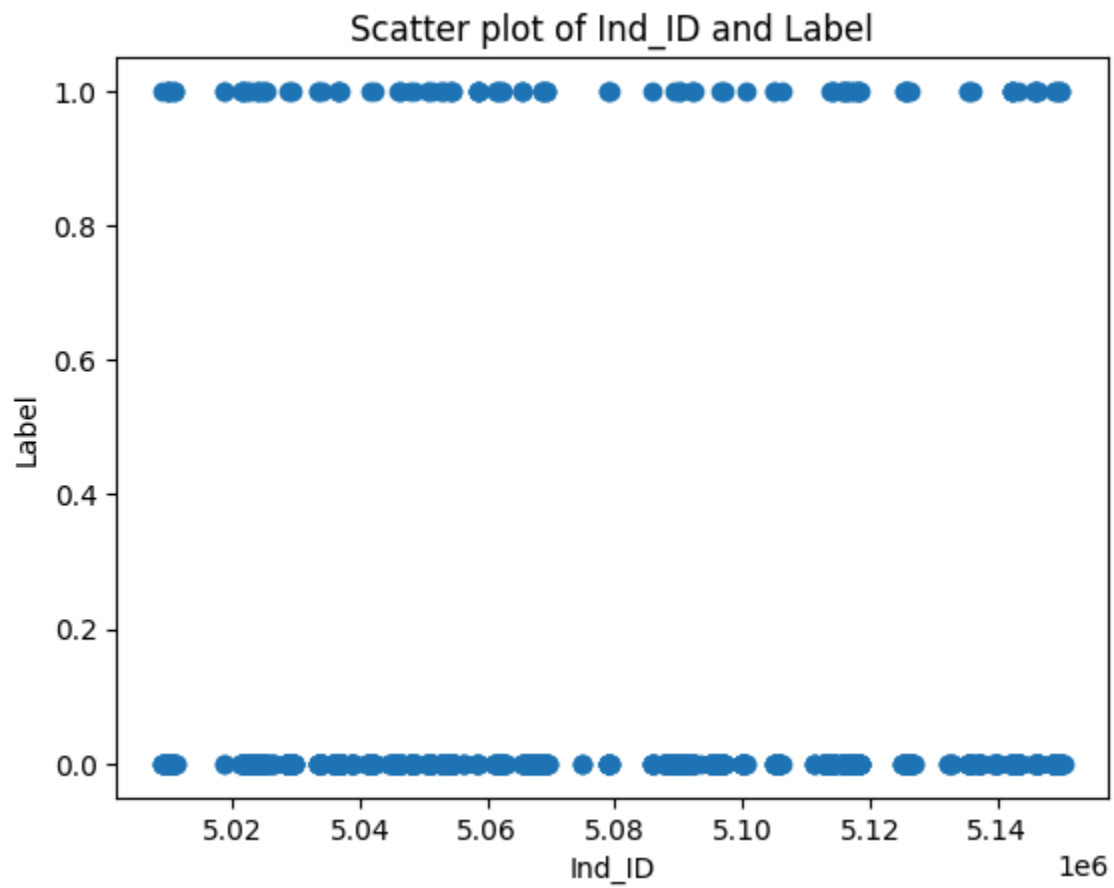
- Bar chart
- Violin plot
- Categorical box plot
- Swarm plot

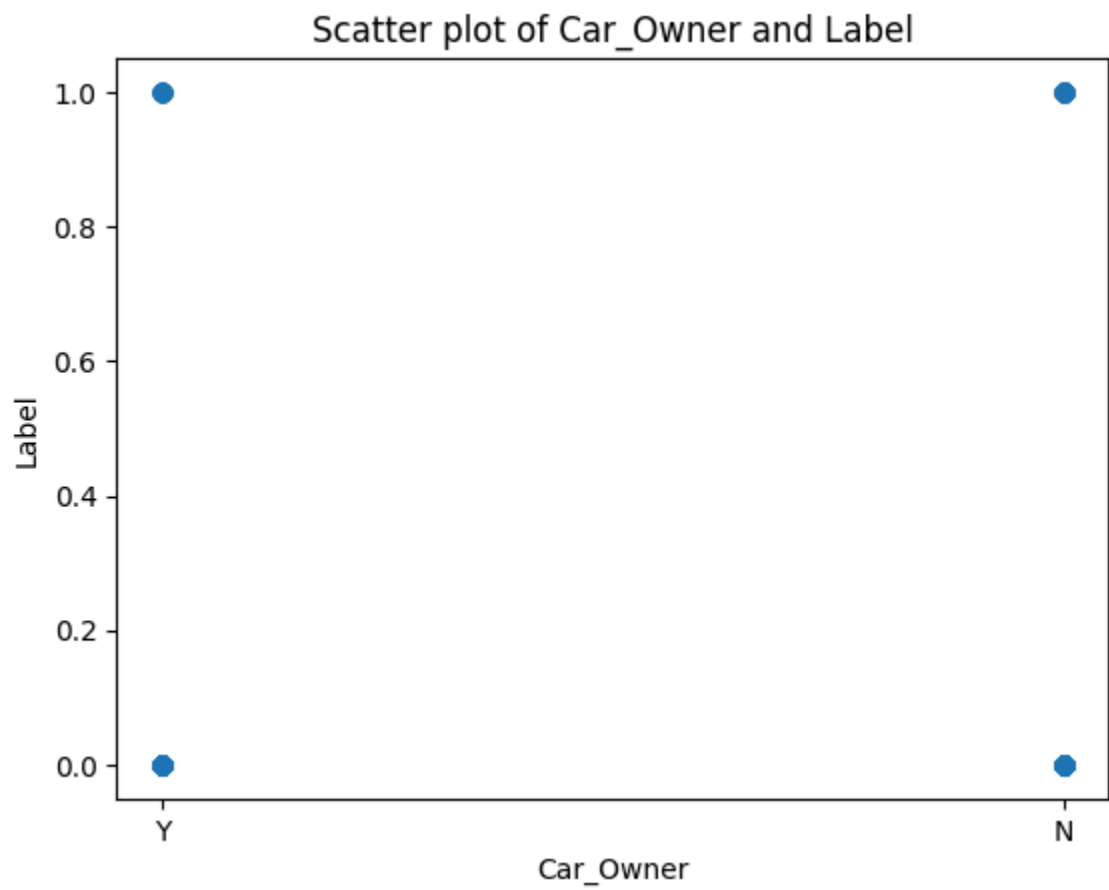
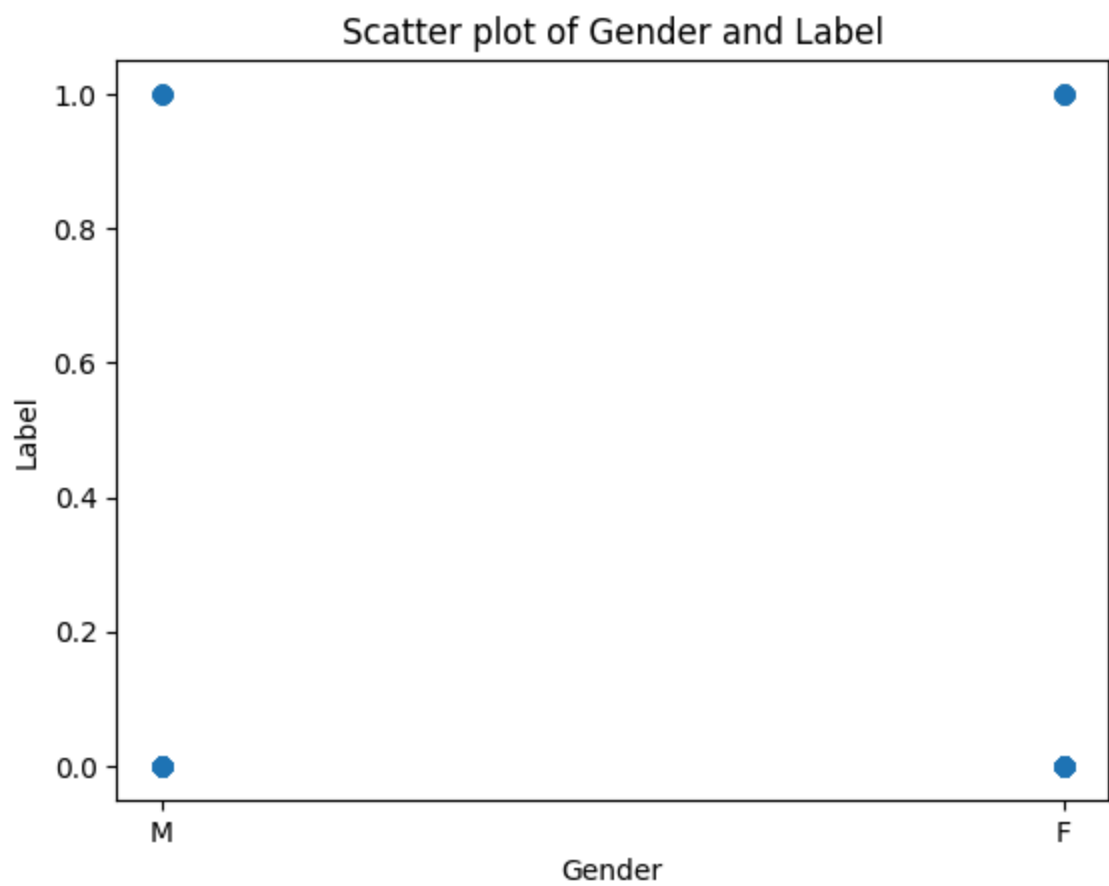
3. Two Categorical Variables

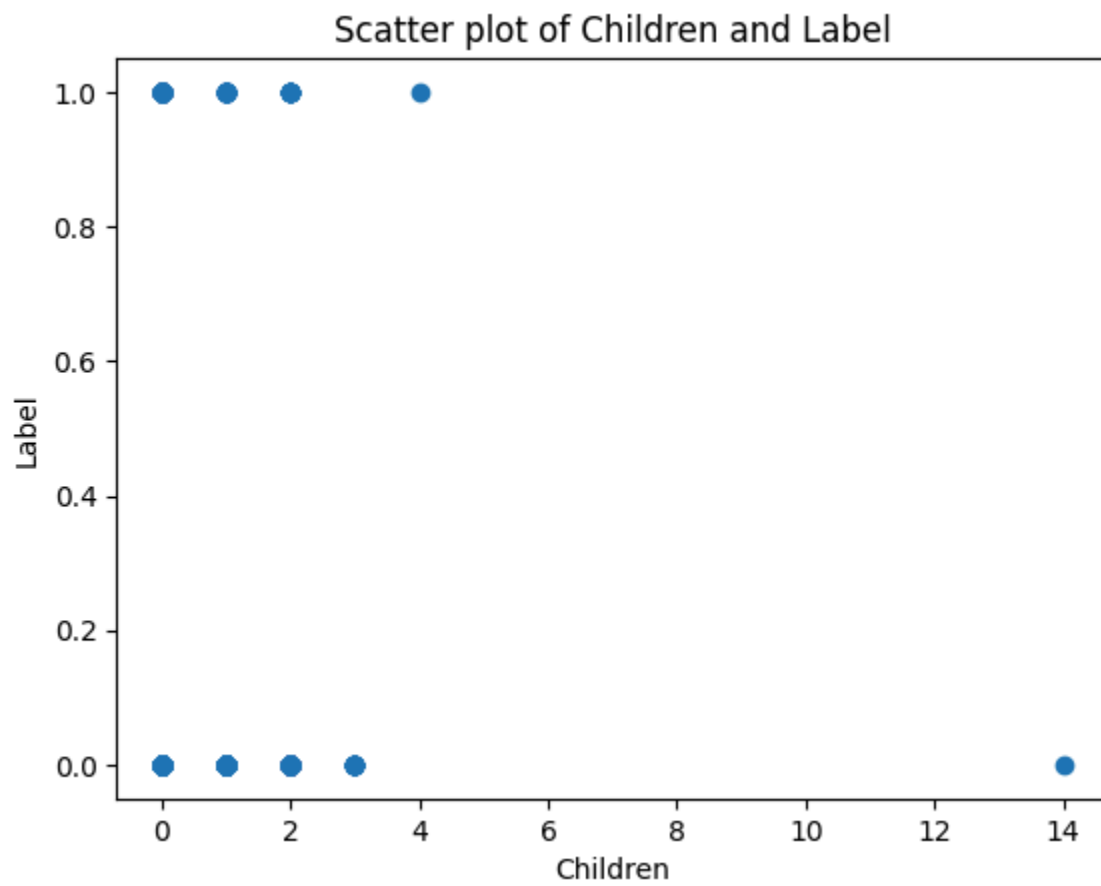
- Bar chart
- Grouped bar chart
- Point plot

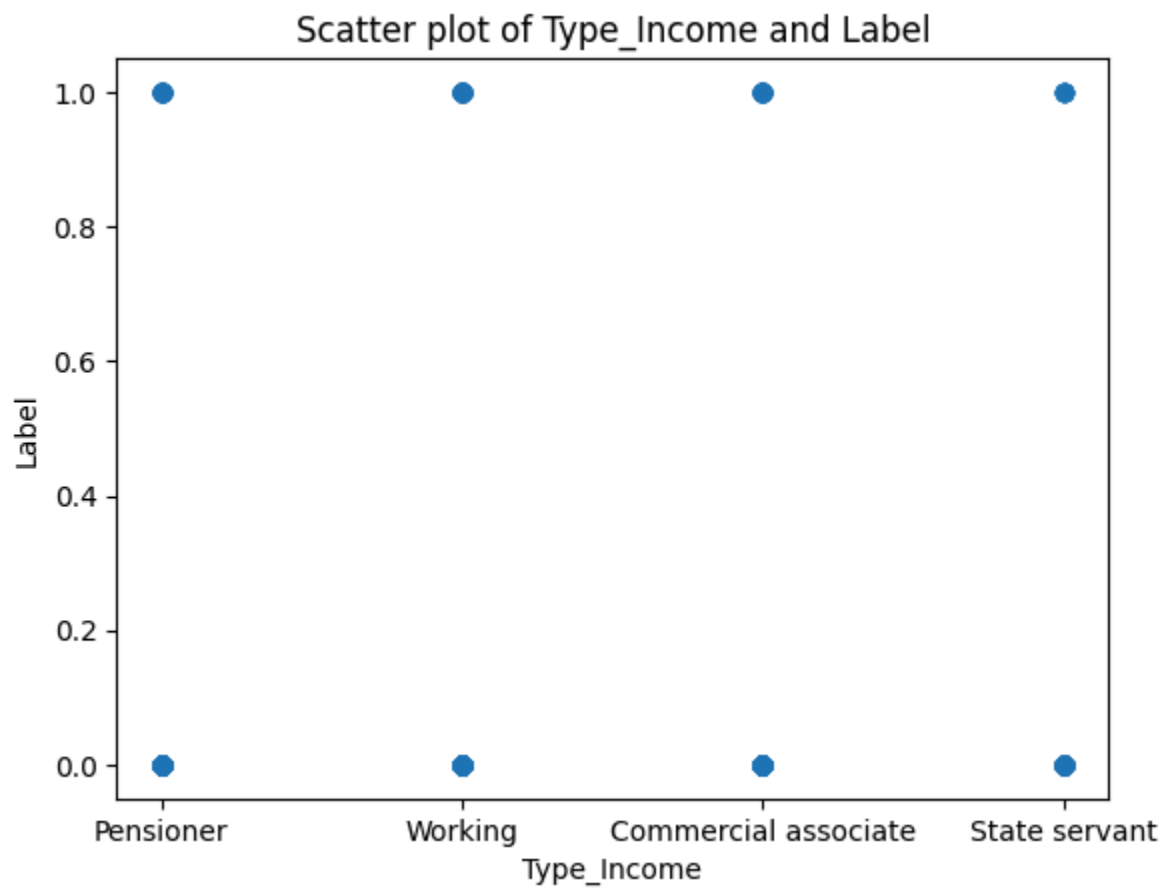
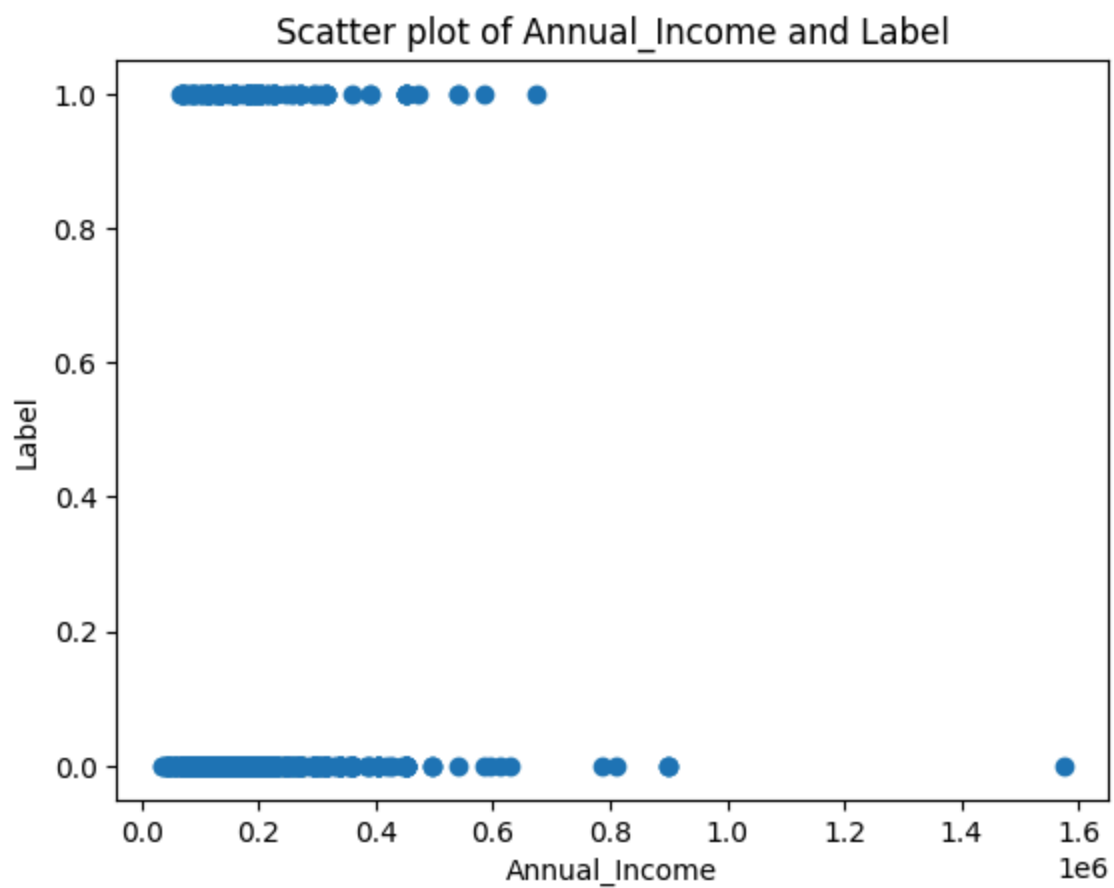
```
In [35]: # numerical vs numerical (scatter plot)
# scatter_var=['Label'] # Rest of the columns vs label willbe plotted
for column in df.columns:
    plt.figure()
    plt.scatter(df[column],df['Label'])
```

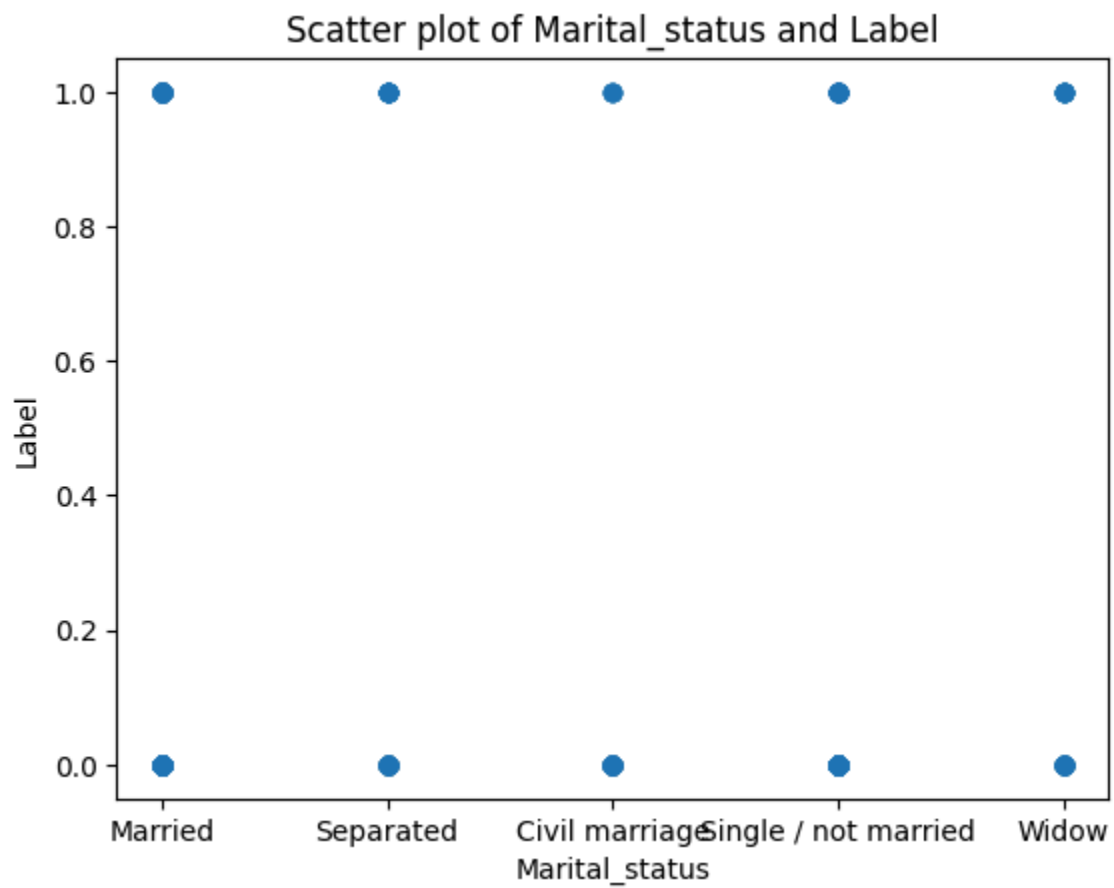
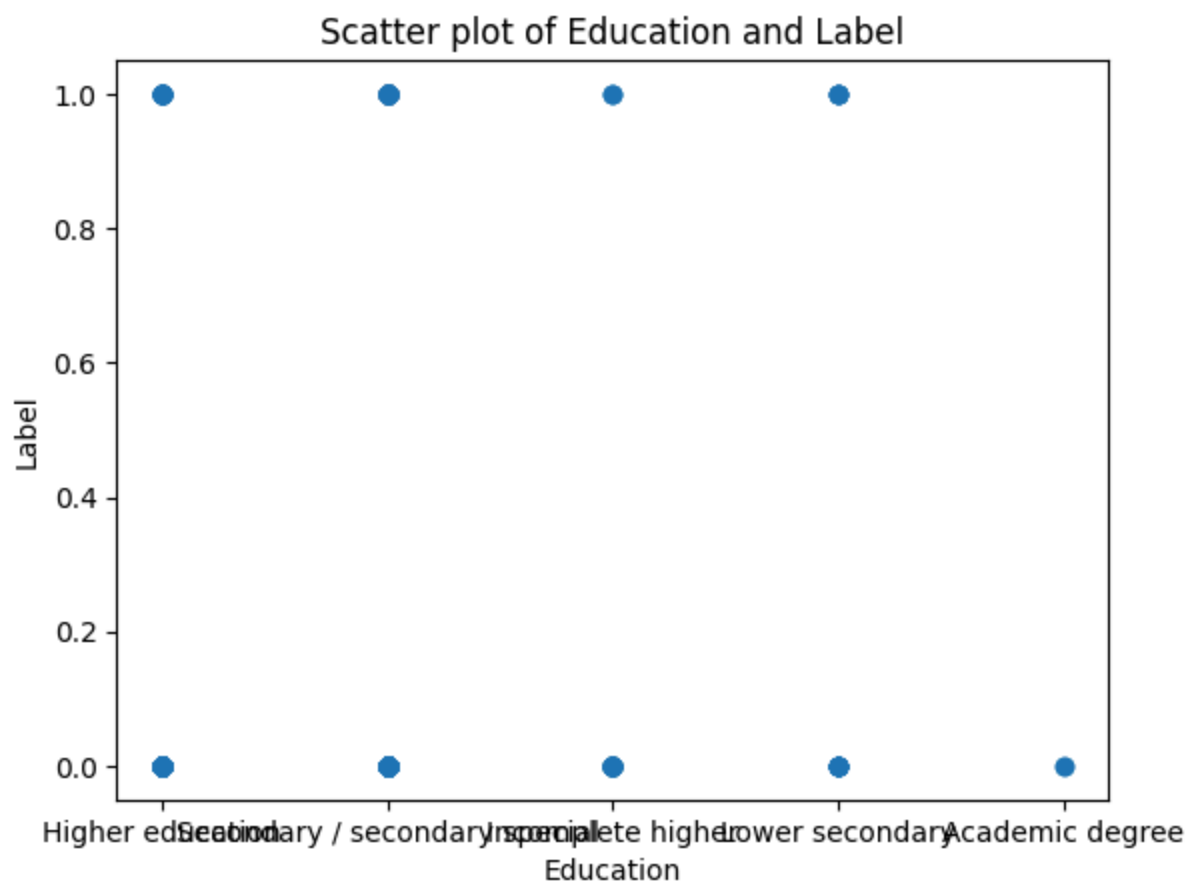
```
plt.title(f'Scatter plot of {column} and Label')  
plt.xlabel(f'{column}')  
plt.ylabel('Label')  
plt.show()
```

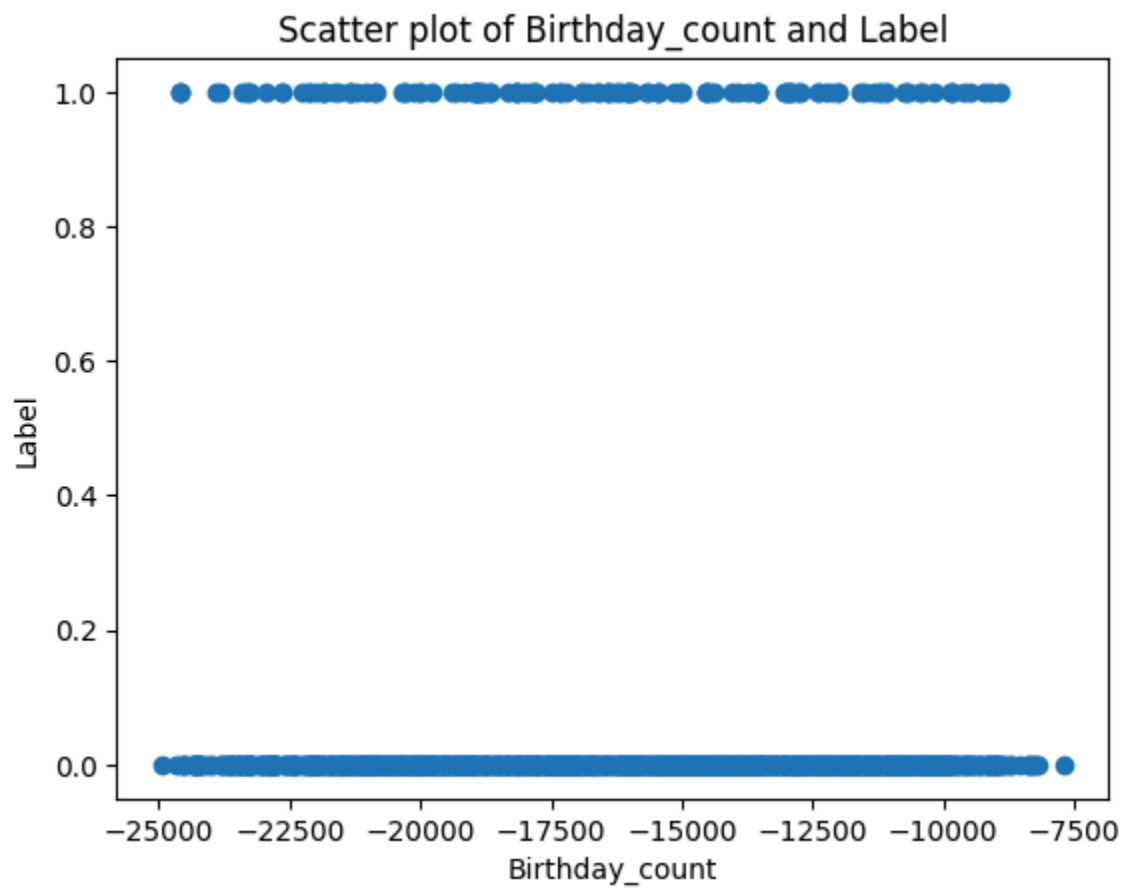
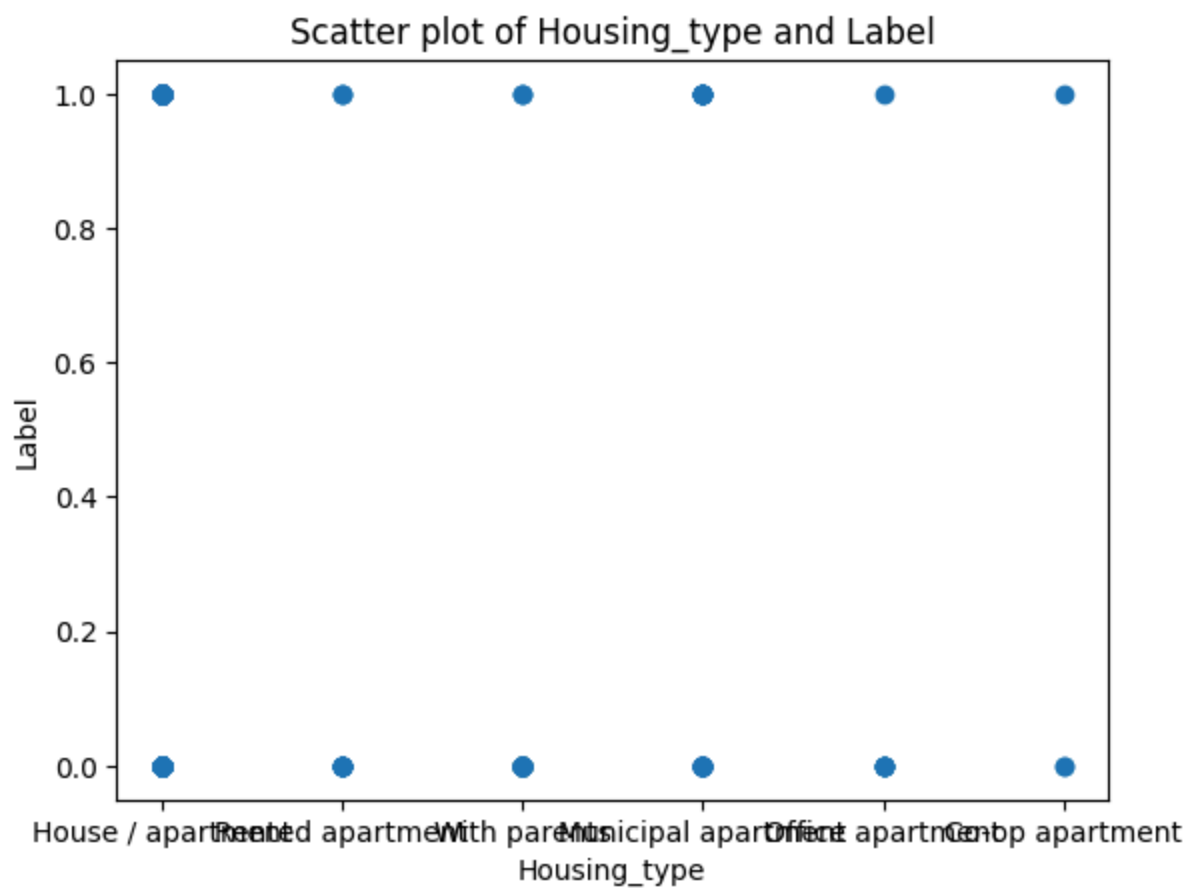


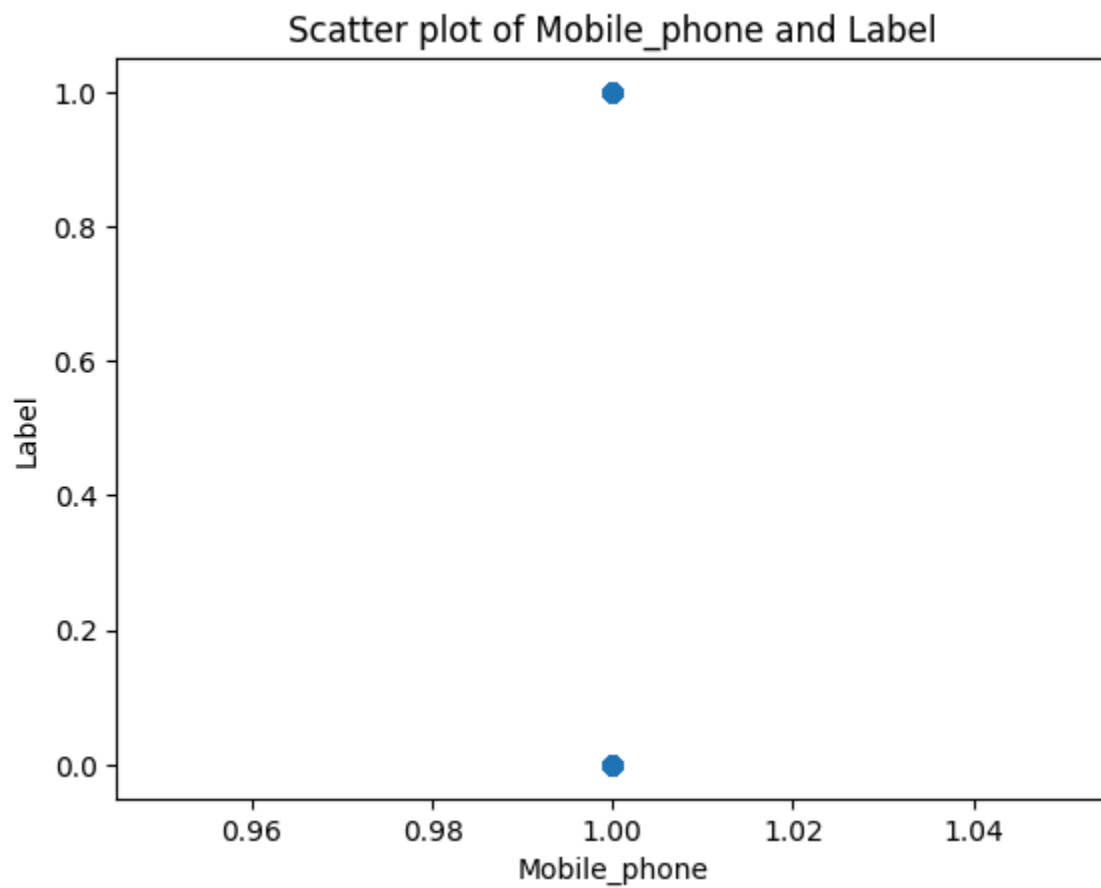
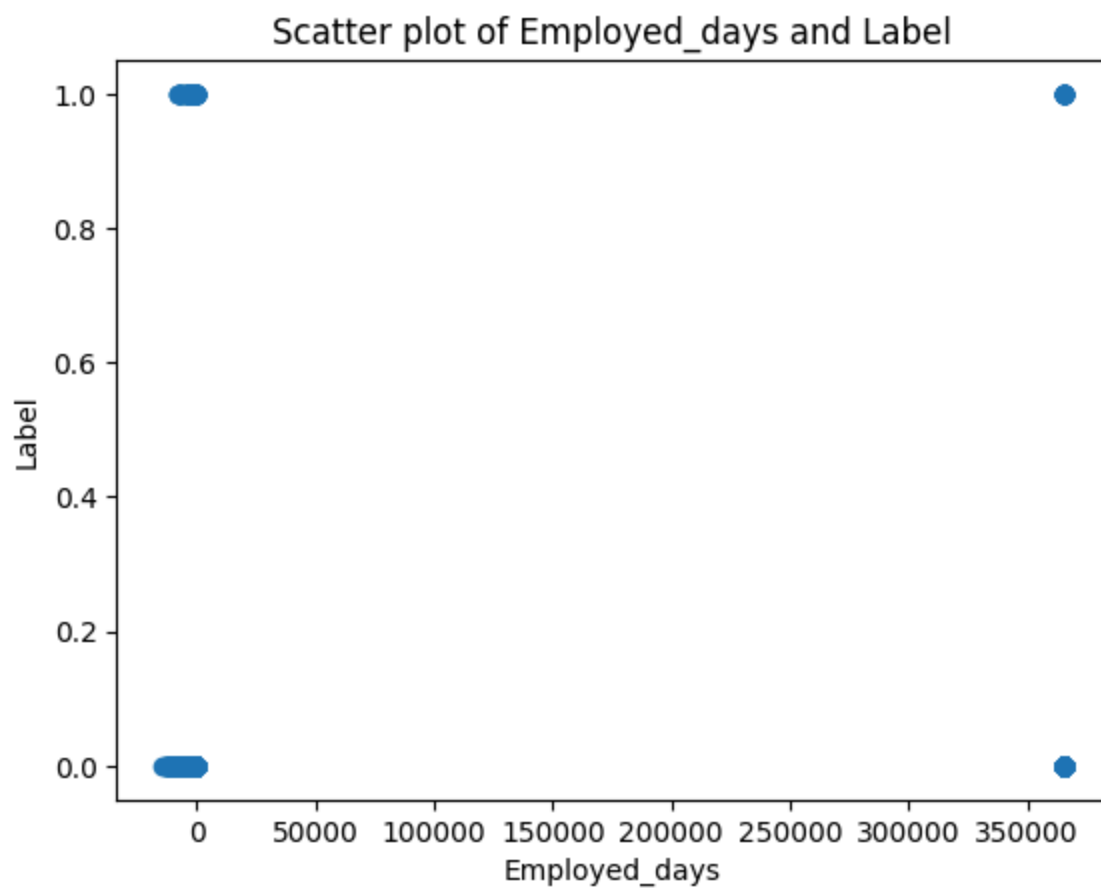




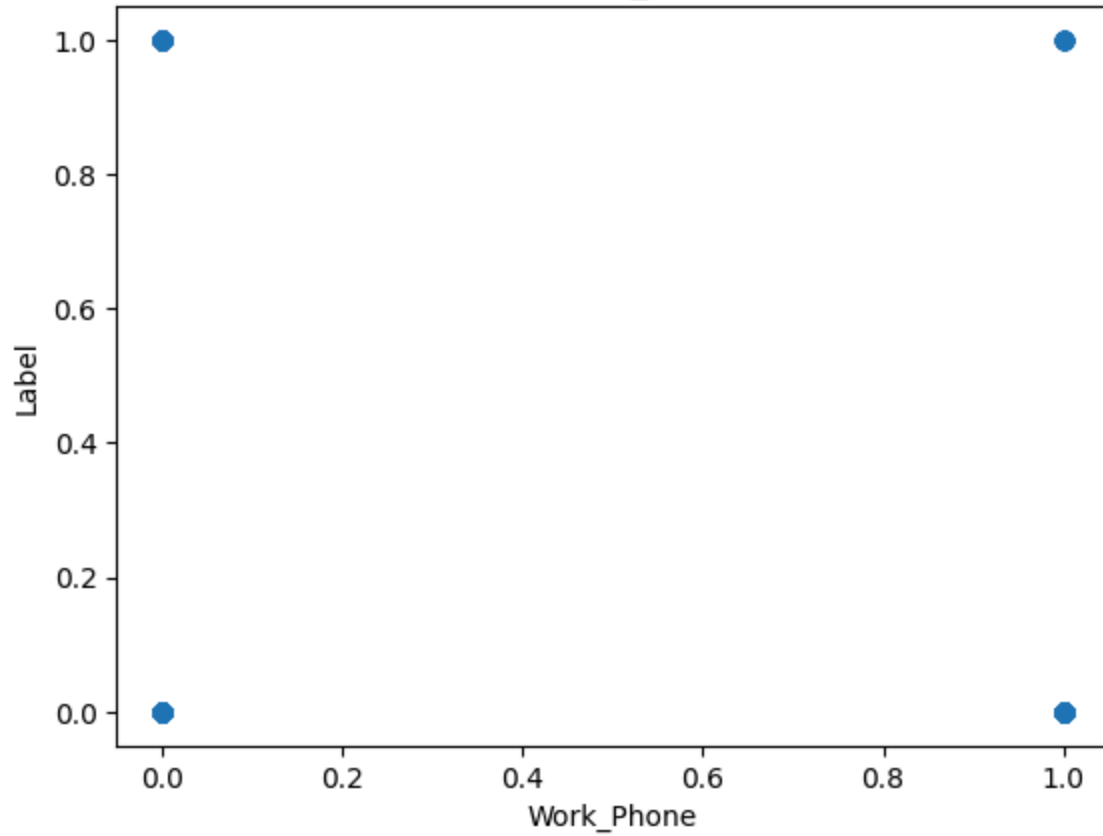




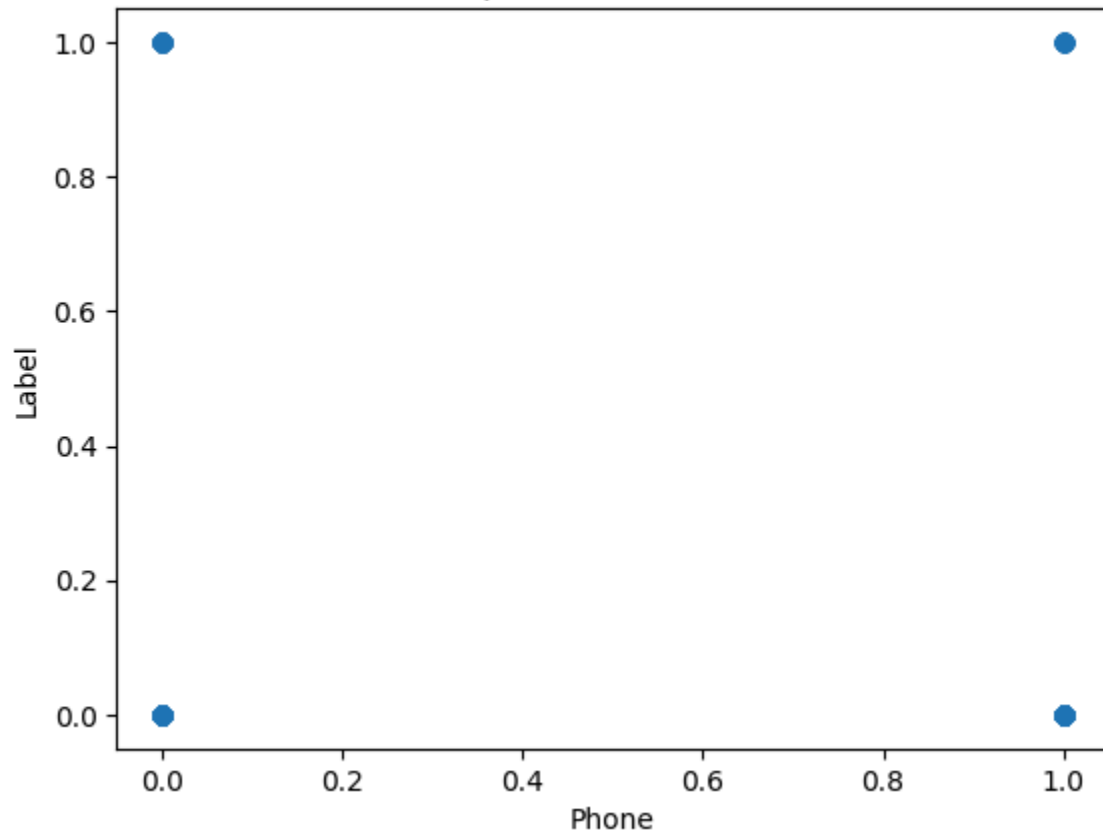




Scatter plot of Work_Phone and Label



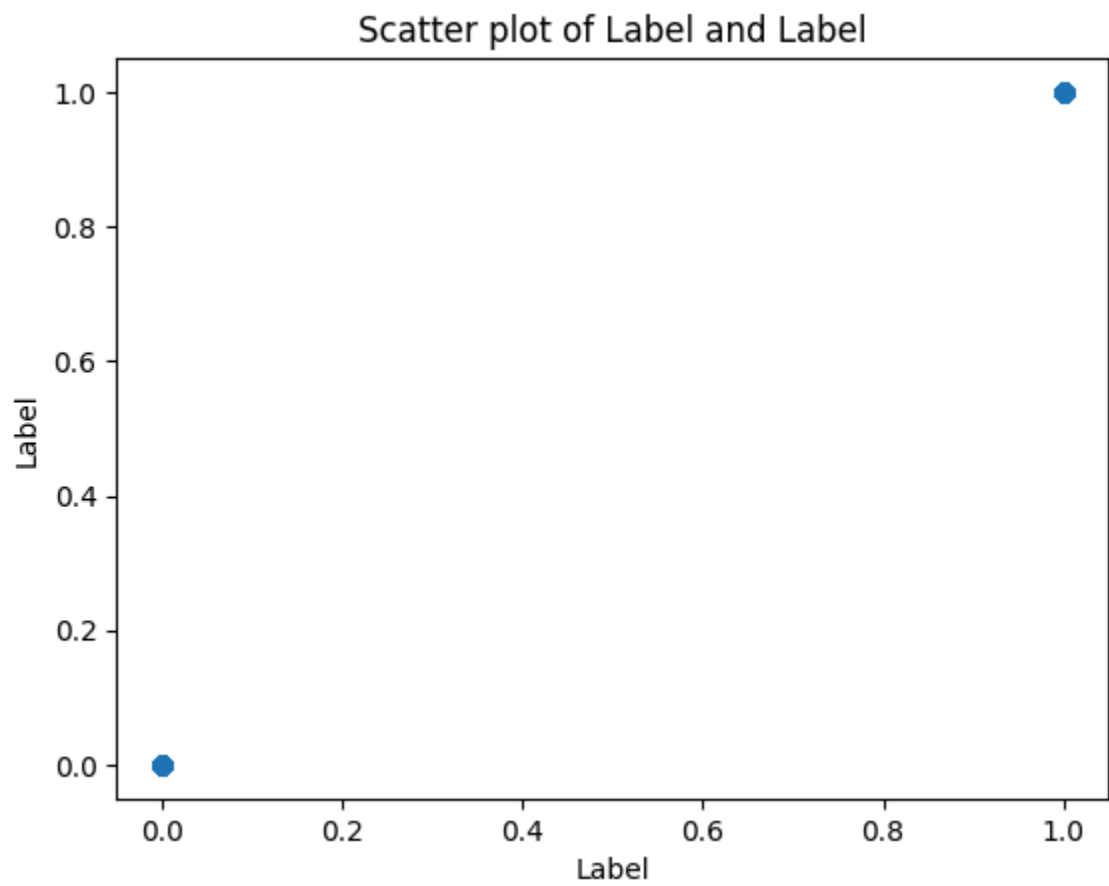
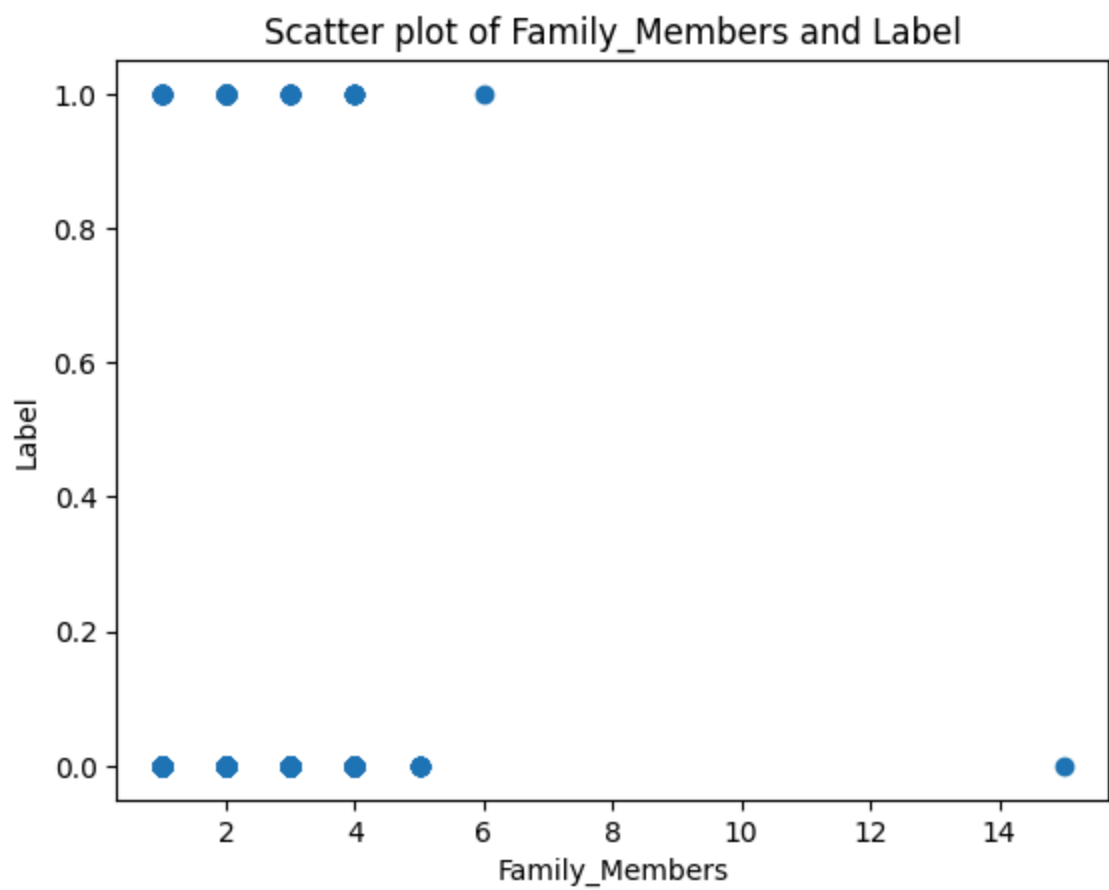
Scatter plot of Phone and Label



A scatter plot showing the relationship between Email_ID and Label. The x-axis, labeled 'Email_ID', has major ticks at 0.0, 0.2, 0.4, 0.6, 0.8, and 1.0. The y-axis, labeled 'Label', has major ticks at 0.0, 0.2, 0.4, 0.6, 0.8, and 1.0. There are four data points plotted as blue circles: one at (0.0, 1.0), one at (0.0, 0.0), one at (1.0, 1.0), and one at (1.0, 0.0).

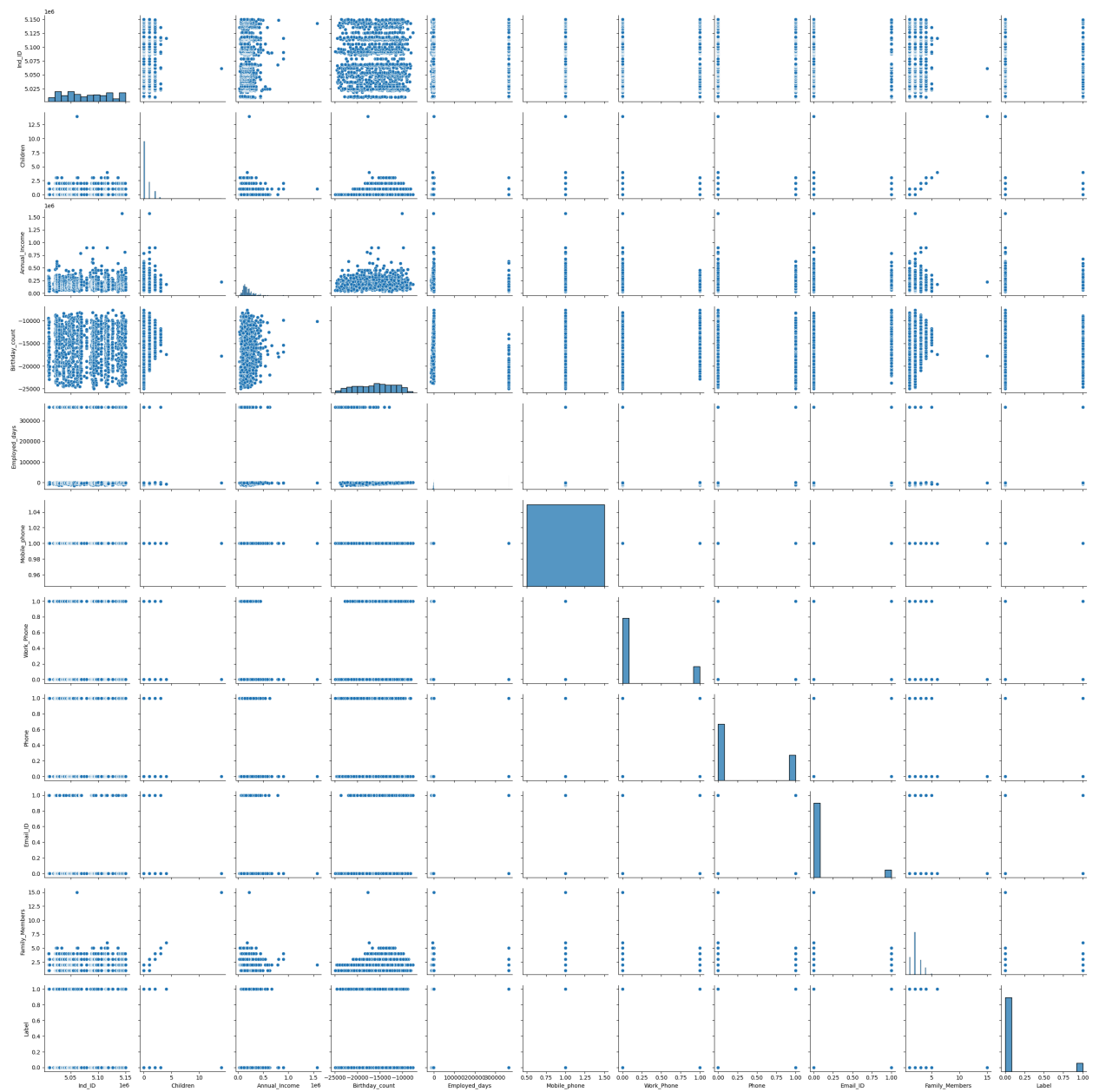
Email_ID	Label
0.0	1.0
0.0	0.0
1.0	1.0
1.0	0.0

Type_Occupation	Label
Lab	1.0
Core_Staff	1.0
Man	1.0
Ac	1.0
Gen	1.0
Plant	1.0
Eng	1.0
Inst	1.0
Stn	1.0
Off	1.0
Ind	1.0
Bus	1.0
Met	1.0
Inf	1.0
W	1.0
Stu	1.0
Wk	1.0
Non_Core_Staff	0.0
Lab	0.0
Core_Staff	0.0
Man	0.0
Ac	0.0
Gen	0.0
Plant	0.0
Eng	0.0
Inst	0.0
Stn	0.0
Off	0.0
Ind	0.0
Bus	0.0
Met	0.0
Inf	0.0
W	0.0
Stu	0.0
Wk	0.0
Non_Core_Staff	0.0



```
In [36]: sns.pairplot(df)
```

```
Out[36]: <seaborn.axisgrid.PairGrid at 0x24fc437a690>
```



```
In [37]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1386 entries, 0 to 1547
Data columns (total 19 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Ind_ID                1386 non-null   int64
1   Gender                1386 non-null   object
2   Car_Owner             1386 non-null   object
3   Property_Owner        1386 non-null   object
4   Children              1386 non-null   int64
5   Annual_Income         1386 non-null   float64
6   Type_Income           1386 non-null   object
7   Education             1386 non-null   object
8   Marital_status        1386 non-null   object
9   Housing_type          1386 non-null   object
10  Birthday_count        1386 non-null   float64
11  Employed_days         1386 non-null   int64
12  Mobile_phone          1386 non-null   int64
13  Work_Phone            1386 non-null   int64
14  Phone                 1386 non-null   int64
15  Email_ID              1386 non-null   int64
16  Type_Occupation        1386 non-null   object
17  Family_Members        1386 non-null   int64
18  Label                 1386 non-null   int64
dtypes: float64(2), int64(9), object(8)
memory usage: 216.6+ KB
```

4. Feature Engineering

Feature engineering involves creating new features or transforming existing features to enhance the performance of machine learning models. It aims to provide the model with more relevant and informative input variables.

(a) Handling Outliers

An Outlier is an observation in a given dataset that lies far from the rest of the observations. That means an outlier is vastly larger or smaller than the remaining values in the set.

Here in this scenario since the annual income is the most important factor, we will remove outliers in the annual income column

```
In [38]: mean = df['Annual_Income'].mean().round(0)
std_dev = df['Annual_Income'].std().round(0)
max = df['Annual_Income'].max().round(0)
min = df['Annual_Income'].min().round(0)
```

```
In [39]: plt.figure(figsize=(20,5))
plt.subplot(1,2,1)
```

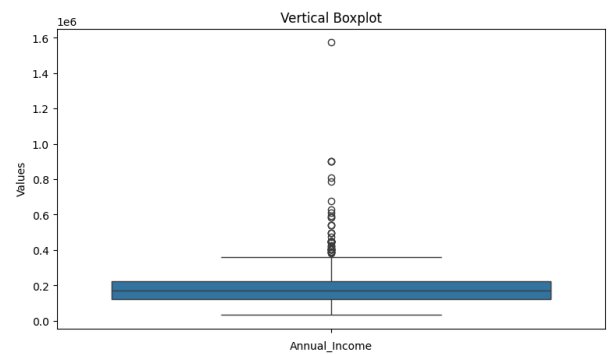
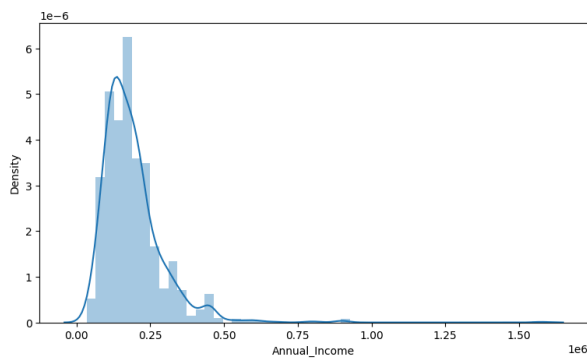
```

sns.distplot(df['Annual_Income'])
print("Skew:",df['Annual_Income'].skew().round(2))
print("Mean:",mean)
print("Std_dev:",std_dev)
print("Max:",max)
print("Min:",min)

plt.subplot(1,2,2)
sns.boxplot(df.Annual_Income)
plt.xlabel("Annual_Income")
plt.ylabel("Values")
plt.title("Vertical Boxplot")
plt.show()

```

Skew: 3.36
 Mean: 189510.0
 Std_dev: 106042.0
 Max: 1575000.0
 Min: 33750.0



```

In [40]: outlier = df[df['Annual_Income']>1000000.0]
outlier

```

```

Out[40]:
   Ind_ID  Gender  Car_Owner  Property_Owner  Children  Annual_Income
1444  5143231      F          Y                Y          1      1575000.0

```

There is one outlier which has an annual income of 1575000 which is disturbing the overall ML model. We will eliminate this outlier.

```

In [41]: df = df.drop(df[df['Annual_Income']> 1000000.0].index)
df.head(4)

```


Out[41]:

	Ind_ID	Gender	Car_Owner	Property_Owner	Children	Annual_Income	Type
0	5008827	M	Y	Y	0	180000.0	
1	5008865	F	Y	Y	2	135000.0	
2	5008889	F	N	Y	0	247500.0	C
3	5009000	M	Y	Y	0	157500.0	

In [42]: `df.shape`

Out[42]: (1385, 19)

In [43]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
Index: 1385 entries, 0 to 1547
Data columns (total 19 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Ind_ID                1385 non-null   int64
1   Gender                1385 non-null   object
2   Car_Owner             1385 non-null   object
3   Property_Owner        1385 non-null   object
4   Children              1385 non-null   int64
5   Annual_Income         1385 non-null   float64
6   Type_Income           1385 non-null   object
7   Education             1385 non-null   object
8   Marital_status        1385 non-null   object
9   Housing_type          1385 non-null   object
10  Birthday_count        1385 non-null   float64
11  Employed_days         1385 non-null   int64
12  Mobile_phone          1385 non-null   int64
13  Work_Phone            1385 non-null   int64
14  Phone                 1385 non-null   int64
15  Email_ID              1385 non-null   int64
16  Type_Occupation       1385 non-null   object
17  Family_Members        1385 non-null   int64
18  Label                 1385 non-null   int64
dtypes: float64(2), int64(9), object(8)
memory usage: 216.4+ KB
```

In [44]:

```
plt.figure(figsize=(20,5))
plt.subplot(1,2,1)
sns.distplot(df['Annual_Income'])
print("Skew:",df['Annual_Income'].skew().round(2))
```

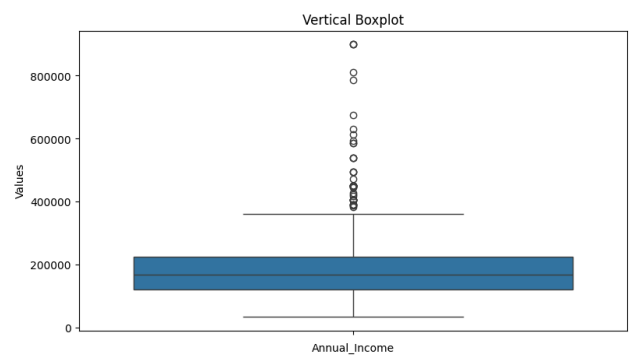
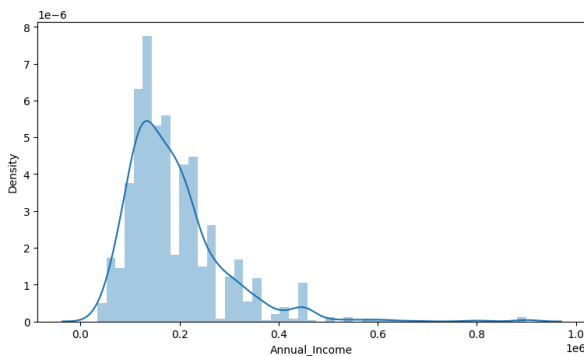
```

print("Mean:",mean)
print("Std_dev:",std_dev)
print("Max:",max)
print("Min:",min)

plt.subplot(1,2,2)
sns.boxplot(df.Annual_Income)
plt.xlabel("Annual_Income")
plt.ylabel("Values")
plt.title("Vertical Boxplot")
plt.show()

```

Skew: 2.15
 Mean: 189510.0
 Std_dev: 106042.0
 Max: 1575000.0
 Min: 33750.0



The annual income is a positively skewed plot with skew=2.15. We have to use the appropriate data transformation techniques like square root, cube root, reciprocal and log to find out which is the best method.

```

In [45]: sqr_root = np.sqrt(df['Annual_Income'])
cube_root = np.cbrt(df['Annual_Income'])
recp = 1/df['Annual_Income']
log_tran = np.log(df['Annual_Income'])

plt.figure(figsize=(20,5))
plt.suptitle("Data transformation of annual income column")

plt.subplot(1,4,1)
sns.distplot(sqr_root)
plt.title("Square root transformation")
print("Skew square root:",sqr_root.skew().round(2))

plt.subplot(1,4,2)
sns.distplot(cube_root)
plt.title("Cube root transformation")
print("Skew cube root:",cube_root.skew().round(2))

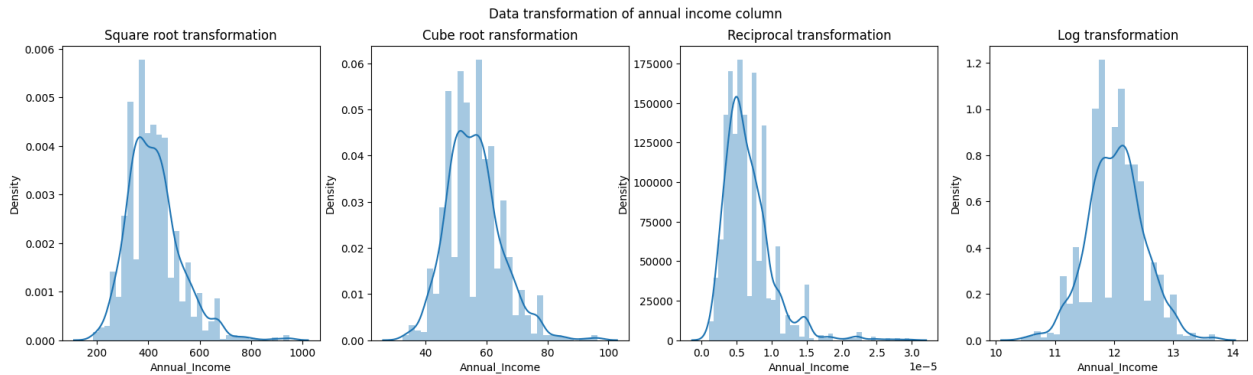
plt.subplot(1,4,3)
sns.distplot(recp)

```

```
plt.title("Reciprocal transformation")
print("Skew reciprocal:", recp.skew().round(2))

plt.subplot(1,4,4)
sns.distplot(log_tran)
plt.title("Log transformation")
print("Skew log:", log_tran.skew().round(2))
```

Skew square root: 0.97
 Skew cube root: 0.65
 Skew reciprocal: 1.86
 Skew log: 0.06



The log transformation technique is the best as the skew= 0.06 which is almost a normal distribution curve

```
In [46]: df['Annual_Income']=log_tran
df.head()
```

```
Out[46]:
```

	Ind_ID	Gender	Car_Owner	Property_Owner	Children	Annual_Income	Type
0	5008827	M	Y	Y	0	12.100712	
1	5008865	F	Y	Y	2	11.813030	
2	5008889	F	N	Y	0	12.419166	C
3	5009000	M	Y	Y	0	11.967181	
4	5009023	F	N	Y	2	12.283034	Sta

(b) Label Encoding for categorical variables

This is required to do since the machine learning algorithms only work on the numerical data. That is why there is a need to convert the categorical column into a numerical one.

label encoder

```
In [47]: from sklearn.preprocessing import LabelEncoder

columns = ['Gender', 'Car_Owner', 'Property_Owner', 'Type_Income', 'Education']

encoder = LabelEncoder()

for column in columns:
    if column in df.columns:
        df[column] = encoder.fit_transform(df[column])
df.head()
```

```
Out[47]:
```

	Ind_ID	Gender	Car_Owner	Property_Owner	Children	Annual_Income	Type
0	5008827	1	1	1	0	12.100712	
1	5008865	0	1	1	2	11.813030	
2	5008889	0	0	1	0	12.419166	
3	5009000	1	1	1	0	11.967181	
4	5009023	0	0	1	2	12.283034	

(c) Feature Scaling

In Data Processing, we try to change the data in such a way that the model can process it without any problems. And Feature Scaling is one such process in which we transform the data into a better version. Feature Scaling is done to normalize the features in the dataset into a finite range.

Lets perform the train-test split on the dataset. To perform a train-test split, use libraries like scikit-learn in Python. Import the `train_test_split` function, specify the dataset, and set the test size (e.g., 20%). This function randomly divides the data into training and testing sets, preserving the distribution of classes or outcomes.

```
In [48]: # Independent variables
X = df.drop(columns=['Label', 'Ind_ID'])
X
```

```
Out[48]:
```

	Gender	Car_Owner	Property_Owner	Children	Annual_Income	Type_Inco
0	1	1	1	0	12.100712	
1	0	1	1	2	11.813030	
2	0	0	1	0	12.419166	
3	1	1	1	0	11.967181	
4	0	0	1	2	12.283034	
...	...	...	...	...	...	...
1543	1	0	0	0	12.100712	
1544	0	0	1	0	12.554967	
1545	0	0	0	0	11.813030	
1546	1	1	1	0	11.662207	
1547	0	0	1	0	11.119883	

1385 rows × 17 columns

```
In [49]: X.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1385 entries, 0 to 1547
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Gender                1385 non-null   int32
1   Car_Owner             1385 non-null   int32
2   Property_Owner        1385 non-null   int32
3   Children              1385 non-null   int64
4   Annual_Income         1385 non-null   float64
5   Type_Income           1385 non-null   int32
6   Education             1385 non-null   int32
7   Marital_status        1385 non-null   int32
8   Housing_type          1385 non-null   int32
9   Birthday_count        1385 non-null   float64
10  Employed_days         1385 non-null   int64
11  Mobile_phone          1385 non-null   int64
12  Work_Phone            1385 non-null   int64
13  Phone                 1385 non-null   int64
14  Email_ID              1385 non-null   int64
15  Type_Occupation        1385 non-null   int32
16  Family_Members        1385 non-null   int64
dtypes: float64(2), int32(8), int64(7)
memory usage: 151.5 KB
```

```
In [50]: # target variable
y = df[['Label']]
y.head()
```

```
Out[50]:
```

	Label
0	1
1	0
2	0
3	0
4	0

```
In [51]: X.shape,y.shape
```

```
Out[51]: ((1385, 17), (1385, 1))
```

Train Test Split for Classification

In classification, the data is split into two parts: training and testing sets. The model is trained on a training set, and its performance is examined on a testing set. The training set contains 80% of the data, whereas the test set contains 20%.

```
In [52]: from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_stat
X_train.shape,X_test.shape
```

```
Out[52]: ((1108, 17), (277, 17))
```

```
In [53]: X_train
```

```
Out[53]:
```

	Gender	Car_Owner	Property_Owner	Children	Annual_Income	Type_Inco
480	0	0	0	0	11.152673	
324	1	1	1	1	11.813030	
1220	0	1	1	0	12.506177	
279	0	0	0	0	11.119883	
644	0	1	0	1	12.218495	
...	...	...	...	...	...	...
848	0	0	0	0	12.660328	
930	1	0	1	0	12.506177	
1358	0	0	0	3	12.100712	
623	0	0	1	2	11.938193	
757	0	1	1	0	12.454884	

1108 rows × 17 columns

In [54]: X_test

Out[54]:

	Gender	Car_Owner	Property_Owner	Children	Annual_Income	Type_Inco
1516	0	0	1	0	12.100712	
1366	0	0	0	0	12.323856	
122	1	1	1	0	12.218495	
1076	0	1	1	1	11.813030	
1151	0	0	1	3	11.630709	
...	...	...	...	...	...	...
638	1	0	1	0	12.218495	
1488	1	1	0	0	12.472276	
444	0	0	0	0	12.162117	
531	0	1	1	0	12.323856	
510	0	0	0	0	12.075394	

277 rows × 17 columns

In [55]: y_train

Out[55]:

	Label
480	0
324	0
1220	0
279	0
644	1
...	...
848	0
930	0
1358	0
623	0
757	1

1108 rows × 1 columns

In [56]: y_test

Out[56]:

	Label
1516	0
1366	0
122	1
1076	0
1151	0
...	...
638	0
1488	0
444	0
531	0
510	0

277 rows × 1 columns

Lets use Standardization technique by using **StandardScaler()**

```
In [57]: # Standardization technique
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()

# Use the fit_transform method to fit the selected columns into the train set,
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

In [58]: X_train_scaled

```
Out[58]: array([[ -0.74870781, -0.80701101, -1.40944064, ..., -0.32355207,
         0.08121139, -1.27953365],
        [ 1.33563453,  1.23914047,  0.70950133, ..., -0.32355207,
         0.08121139,  0.91678438],
        [ -0.74870781,  1.23914047,  0.70950133, ..., -0.32355207,
         1.81720277, -1.27953365],
        ...,
        [ -0.74870781, -0.80701101, -1.40944064, ..., -0.32355207,
        -1.36544809,  3.11310241],
        [ -0.74870781, -0.80701101,  0.70950133, ..., -0.32355207,
        -2.23344378,  2.01494339],
        [ -0.74870781,  1.23914047,  0.70950133, ..., -0.32355207,
        -2.23344378, -0.18137464]])
```

In [59]: X_test_scaled


```
Out[59]: array([[ -0.74870781, -0.80701101,  0.70950133, ..., -0.32355207,
        0.08121139, -0.18137464],
       [ -0.74870781, -0.80701101, -1.40944064, ..., -0.32355207,
        0.08121139, -0.18137464],
       [  1.33563453,  1.23914047,  0.70950133, ...,  3.09069263,
        0.08121139, -0.18137464],
       ...,
       [ -0.74870781, -0.80701101, -1.40944064, ..., -0.32355207,
        -1.36544809, -1.27953365],
       [ -0.74870781,  1.23914047,  0.70950133, ..., -0.32355207,
        -0.4974524 , -1.27953365],
       [ -0.74870781, -0.80701101, -1.40944064, ..., -0.32355207,
        -0.4974524 , -1.27953365]])
```

```
In [60]: # both the X_Train and X_test should be converted into dataframe for easy read
X_train_scaled = pd.DataFrame(X_train_scaled, columns=X_train.columns)
X_test_scaled = pd.DataFrame(X_test_scaled, columns=X_test.columns)
```

```
In [61]: X_train_scaled
```

```
Out[61]:
```

	Gender	Car_Owner	Property_Owner	Children	Annual_Income	Type_In
0	-0.748708	-0.807011	-1.409441	-0.589622	-1.890338	-0.6
1	1.335635	1.239140	0.709501	0.809311	-0.479192	0.8
2	-0.748708	1.239140	0.709501	-0.589622	1.002025	-1.4
3	-0.748708	-0.807011	-1.409441	-0.589622	-1.960408	-1.4
4	-0.748708	1.239140	-1.409441	0.809311	0.387264	0.8
...	...	...	...	...	...	...
1103	-0.748708	-0.807011	-1.409441	-0.589622	1.331436	-1.4
1104	1.335635	-0.807011	0.709501	-0.589622	1.002025	-1.4
1105	-0.748708	-0.807011	-1.409441	3.607177	0.135569	0.8
1106	-0.748708	-0.807011	0.709501	2.208244	-0.211725	-1.4
1107	-0.748708	1.239140	0.709501	-0.589622	0.892414	-1.4

1108 rows × 7 columns

```
In [62]: X_test_scaled
```

```
Out[62]:
```

	Gender	Car_Owner	Property_Owner	Children	Annual_Income	Type_Inc
0	-0.748708	-0.807011	0.709501	-0.589622	0.135569	0.87
1	-0.748708	-0.807011	-1.409441	-0.589622	0.612414	-0.69
2	1.335635	1.239140	0.709501	-0.589622	0.387264	-1.48
3	-0.748708	1.239140	0.709501	0.809311	-0.479192	0.87
4	-0.748708	-0.807011	0.709501	3.607177	-0.868803	0.87
...	...	...	...	...	...	...
272	1.335635	-0.807011	0.709501	-0.589622	0.387264	-1.48
273	1.335635	1.239140	-1.409441	-0.589622	0.929579	-1.48
274	-0.748708	-0.807011	-1.409441	-0.589622	0.266788	0.87
275	-0.748708	1.239140	0.709501	-0.589622	0.612414	-1.48
276	-0.748708	-0.807011	-1.409441	-0.589622	0.081466	0.87

277 rows × 7 columns

```
In [63]: # to check if the mean = 0 and std = 1 for all columns
X_train_scaled.describe().round()
```

```
Out[63]:
```

	Gender	Car_Owner	Property_Owner	Children	Annual_Income	Type_Inc
count	1108.0	1108.0	1108.0	1108.0	1108.0	1108.0
mean	-0.0	-0.0	-0.0	0.0	-0.0	-0.0
std	1.0	1.0	1.0	1.0	1.0	1.0
min	-1.0	-1.0	-1.0	-1.0	-3.0	-1.0
25%	-1.0	-1.0	-1.0	-1.0	-1.0	-1.0
50%	-1.0	-1.0	1.0	-1.0	-0.0	-1.0
75%	1.0	1.0	1.0	1.0	1.0	1.0
max	1.0	1.0	1.0	4.0	4.0	1.0

```
In [64]: X_test_scaled.describe().round()
```

Out[64]:

	Gender	Car_Owner	Property_Owner	Children	Annual_Income	Type_Income
count	277.0	277.0	277.0	277.0	277.0	277.0
mean	0.0	0.0	-0.0	0.0	-0.0	-0.0
std	1.0	1.0	1.0	1.0	1.0	1.0
min	-1.0	-1.0	-1.0	-1.0	-3.0	-3.0
25%	-1.0	-1.0	-1.0	-1.0	-1.0	-1.0
50%	-1.0	-1.0	1.0	-1.0	0.0	0.0
75%	1.0	1.0	1.0	1.0	1.0	1.0
max	1.0	1.0	1.0	19.0	3.0	3.0

In [65]: `X_train_scaled.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1108 entries, 0 to 1107
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Gender                 1108 non-null   float64
1   Car_Owner              1108 non-null   float64
2   Property_Owner         1108 non-null   float64
3   Children               1108 non-null   float64
4   Annual_Income          1108 non-null   float64
5   Type_Income            1108 non-null   float64
6   Education              1108 non-null   float64
7   Marital_status         1108 non-null   float64
8   Housing_type           1108 non-null   float64
9   Birthday_count         1108 non-null   float64
10  Employed_days          1108 non-null   float64
11  Mobile_phone           1108 non-null   float64
12  Work_Phone             1108 non-null   float64
13  Phone                  1108 non-null   float64
14  Email_ID               1108 non-null   float64
15  Type_Occupation        1108 non-null   float64
16  Family_Members         1108 non-null   float64
dtypes: float64(17)
memory usage: 147.3 KB
```

(d) Feature Selection

The goal of feature selection techniques in machine learning is to find the best set of features that allows one to build optimized models of studied phenomena.

- Filter Methods: Select features based on statistical metrics like correlation or mutual information.
- Wrapper Methods: Use specific machine learning algorithms to evaluate

subsets of features.

- Embedded Methods: Feature selection is inherent to the model training process, such as LASSO regression.

5. Machine Learning Model

we will use several machine learning algorithms and find out which is the best ML algorithm.

In ML, depending on the problem it is either a classification or a regression.

We have a binary classification problem.

We shall use the following methods:-

1. Logistic regression
2. Support vector machine (SVM) algorithm
3. Random forest algorithm
4. Decision tree algorithm
5. K-Nearest Neighbour (KNN) algorithm
6. Gradient boost classifier

In a binary classification problem like credit card approval, where Type 1 error (false positives) is considered dangerous, we should prioritize precision over other metrics. A high precision ensures that when the model predicts positive cases (credit card approval), it is likely correct.

For precision, the parameters average = 'binary' and zero_division = 1

So the ML algorithm with the highest precision score will be considered as the best method.

```
In [82]: # we will import the accuracy, precision, confusion matrix and classification
from sklearn.metrics import accuracy_score, precision_score, recall_score, cla
```

1. Logistic Regression

```
In [117]: from sklearn.linear_model import LogisticRegression

# assign a logistic model to the variable **logistic_model**
logistic_model = LogisticRegression()

# train the model using the training datasets
```

```

logistic_model.fit(X_train_scaled,y_train)

# predict the test data
y_prediction_log = logistic_model.predict(X_test_scaled)

accuracy_log = accuracy_score(y_test,y_prediction_log)
precision_score_log = precision_score(y_test,y_prediction_log,average='binary')
confusion_matrix_log = confusion_matrix(y_test,y_prediction_log)
classification_report_log = classification_report(y_test,y_prediction_log)
recall_score_log = recall_score(y_test,y_prediction_log)
f1_score_log = f1_score(y_test,y_prediction_log)

print("Confusion matrix:\n",confusion_matrix_log,"\n")
print("Accuracy:",round(accuracy_log*100,2),"%\n")
print("Precision:",round(precision_score_log*100,2),"%\n")
print("Recall score:",round(recall_score_log*100,2),"%\n")
print("f1 score:",round(f1_score_log*100,2),"%\n")
print(classification_report_log)

```

Confusion matrix:

```

[[250   0]
 [ 27   0]]

```

Accuracy: 90.25 %

Precision: 100.0 %

Recall score: 0.0 %

f1 score: 0.0 %

	precision	recall	f1-score	support
0	0.90	1.00	0.95	250
1	0.00	0.00	0.00	27
accuracy			0.90	277
macro avg	0.45	0.50	0.47	277
weighted avg	0.81	0.90	0.86	277

2. Support Vector Machines (SVMs)

In [118...

```

from sklearn import svm

svm_model = svm.SVC(kernel='linear')

svm_model.fit(X_train_scaled,y_train)

svm_y_prediction = svm_model.predict(X_test_scaled)

accuracy_svm = accuracy_score(y_test,svm_y_prediction)

```

```

precision_score_svm = precision_score(y_test,svm_y_prediction,average='binary')
confusion_matrix_svm = confusion_matrix(y_test,svm_y_prediction)
classification_report_svm = classification_report(y_test,svm_y_prediction)
recall_score_svm = recall_score(y_test,svm_y_prediction)
f1_score_svm= f1_score(y_test,svm_y_prediction)

print("Confusion matrix:\n",confusion_matrix_svm,"\n")
print("Accuracy:",round(accuracy_svm*100,2),"%\n")
print("Precision:",round(precision_score_svm*100,2),"%\n")
print("Recall score:",round(recall_score_svm*100,2),"%\n")
print("f1 score:",round(f1_score_svm*100,2),"%\n")
print(classification_report_svm)

```

Confusion matrix:

```

[[250   0]
 [ 27   0]]

```

Accuracy: 90.25 %

Precision: 100.0 %

Recall score: 0.0 %

f1 score: 0.0 %

	precision	recall	f1-score	support
0	0.90	1.00	0.95	250
1	0.00	0.00	0.00	27
accuracy			0.90	277
macro avg	0.45	0.50	0.47	277
weighted avg	0.81	0.90	0.86	277

3. Random Forest

```

In [119... from sklearn.ensemble import RandomForestClassifier

random_model = RandomForestClassifier(n_estimators=100,criterion='entropy')

random_model.fit(X_train_scaled,y_train)

randomf_y_prediction = random_model.predict(X_test_scaled)

accuracy_rf = accuracy_score(y_test,randomf_y_prediction)
precision_score_rf = precision_score(y_test,randomf_y_prediction,average='binary')
confusion_matrix_rf = confusion_matrix(y_test,randomf_y_prediction)
classification_report_rf = classification_report(y_test,randomf_y_prediction)
recall_score_rf = recall_score(y_test,randomf_y_prediction)
f1_score_rf= f1_score(y_test,randomf_y_prediction)

print("Confusion matrix:\n",confusion_matrix_rf,"\n")

```

```

print("Accuracy:",round(accuracy_rf*100,2),"%\n")
print("Precision:",round(precision_score_rf*100,2),"%\n")
print("Recall score:",round(recall_score_rf*100,2),"%\n")
print("f1 score:",round(f1_score_rf*100,2),"%\n")
print(classification_report_rf)

```

Confusion matrix:

```

[[245   5]
 [ 22   5]]

```

Accuracy: 90.25 %

Precision: 50.0 %

Recall score: 18.52 %

f1 score: 27.03 %

	precision	recall	f1-score	support
0	0.92	0.98	0.95	250
1	0.50	0.19	0.27	27
accuracy			0.90	277
macro avg	0.71	0.58	0.61	277
weighted avg	0.88	0.90	0.88	277

4. Gradient Boosting

```

In [120... from sklearn.ensemble import GradientBoostingClassifier

gradient_model = GradientBoostingClassifier(n_estimators=100, random_state=42)

gradient_model.fit(X_train_scaled,y_train)

gradient_y_prediction = gradient_model.predict(X_test_scaled)

accuracy_gradient = accuracy_score(y_test,gradient_y_prediction)
precision_score_gradient = precision_score(y_test,gradient_y_prediction,average='macro')
confusion_matrix_gradient = confusion_matrix(y_test,gradient_y_prediction)
classification_report_gradient = classification_report(y_test,gradient_y_prediction)
recall_score_gradient = recall_score(y_test,gradient_y_prediction)
f1_score_gradient= f1_score(y_test,gradient_y_prediction)

print("Confusion matrix:\n",confusion_matrix_gradient,"\n")
print("Accuracy:",round(accuracy_gradient*100,2),"%\n")
print("Precision:",round(precision_score_gradient*100,2),"%\n")
print("Recall score:",round(recall_score_gradient*100,2),"%\n")
print("f1 score:",round(f1_score_gradient*100,2),"%\n")
print(classification_report_gradient)

```

Confusion matrix:

```
[[246  4]
 [ 23  4]]
```

Accuracy: 90.25 %

Precision: 50.0 %

Recall score: 14.81 %

f1 score: 22.86 %

	precision	recall	f1-score	support
0	0.91	0.98	0.95	250
1	0.50	0.15	0.23	27
accuracy			0.90	277
macro avg	0.71	0.57	0.59	277
weighted avg	0.87	0.90	0.88	277

5. Decision Tree

```
In [121... from sklearn import tree

dt_model = tree.DecisionTreeClassifier()

dt_model.fit(X_train_scaled,y_train)

y_prediction_dt = dt_model.predict(X_test_scaled)

accuracy_dt = accuracy_score(y_test,y_prediction_dt)
precision_score_dt = precision_score(y_test,y_prediction_dt,average='binary',
confusion_matrix_dt = confusion_matrix(y_test,y_prediction_dt)
classification_report_dt = classification_report(y_test,y_prediction_dt)
recall_score_dt = recall_score(y_test,y_prediction_dt)
f1_score_dt = f1_score(y_test,y_prediction_dt)

print("Confusion matrix:\n",confusion_matrix_dt,"\n")
print("Accuracy:",round(accuracy_dt*100,2),"%\n")
print("Precision:",round(precision_score_dt*100,2),"%\n")
print("Recall score:",round(recall_score_dt*100,2),"%\n")
print("f1 score:",round(f1_score_dt*100,2),"%\n")
print(classification_report_dt)
```


Confusion matrix:

```
[[223  27]
 [ 19   8]]
```

Accuracy: 83.39 %

Precision: 22.86 %

Recall score: 29.63 %

f1 score: 25.81 %

	precision	recall	f1-score	support
0	0.92	0.89	0.91	250
1	0.23	0.30	0.26	27
accuracy			0.83	277
macro avg	0.58	0.59	0.58	277
weighted avg	0.85	0.83	0.84	277

6. K-Nearest Neighbours (KNN)

```
In [122... from sklearn.neighbors import KNeighborsClassifier

knn_model = KNeighborsClassifier()

knn_model.fit(X_train_scaled,y_train)

y_prediction_knn = knn_model.predict(X_test_scaled)

accuracy_knn = accuracy_score(y_test,y_prediction_knn)
precision_score_knn = precision_score(y_test,y_prediction_knn,average='binary')
confusion_matrix_knn = confusion_matrix(y_test,y_prediction_knn)
classification_report_knn = classification_report(y_test,y_prediction_knn)
recall_score_knn = recall_score(y_test,y_prediction_knn)
f1_score_knn = f1_score(y_test,y_prediction_knn)

print("Confusion matrix:\n",confusion_matrix_knn,"\n")
print("Accuracy:",round(accuracy_knn*100,2), "%\n")
print("Precision:",round(precision_score_knn*100,2), "%\n")
print("Recall score:",round(recall_score_knn*100,2), "%\n")
print("f1 score:",round(f1_score_knn*100,2), "%\n")
print(classification_report_knn)
```

Confusion matrix:

```
[[248  2]
 [ 27  0]]
```

Accuracy: 89.53 %

Precision: 0.0 %

Recall score: 0.0 %

f1 score: 0.0 %

	precision	recall	f1-score	support
0	0.90	0.99	0.94	250
1	0.00	0.00	0.00	27
accuracy			0.90	277
macro avg	0.45	0.50	0.47	277
weighted avg	0.81	0.90	0.85	277

7. Gaussian Naive Bayes

```
In [123... from sklearn.naive_bayes import GaussianNB

gnb_model = GaussianNB()

gnb_model.fit(X_train_scaled,y_train)

y_prediction_gnb = gnb_model.predict(X_test_scaled)

accuracy_gnb = accuracy_score(y_test,y_prediction_gnb)
precision_score_gnb = precision_score(y_test,y_prediction_gnb,average='binary')
confusion_matrix_gnb = confusion_matrix(y_test,y_prediction_gnb)
classification_report_gnb = classification_report(y_test,y_prediction_gnb)
recall_score_gnb = recall_score(y_test,y_prediction_gnb)
f1_score_gnb = f1_score(y_test,y_prediction_gnb)

print("Confusion matrix:\n",confusion_matrix_gnb,"\n")
print("Accuracy:",round(accuracy_gnb*100,2), "%\n")
print("Precision:",round(precision_score_gnb*100,2), "%\n")
print("Recall score:",round(recall_score_gnb*100,2), "%\n")
print("f1 score:",round(f1_score_gnb*100,2), "%\n")
print(classification_report_gnb)
```

Confusion matrix:

```
[[244  6]
 [ 25  2]]
```

Accuracy: 88.81 %

Precision: 25.0 %

Recall score: 7.41 %

f1 score: 11.43 %

	precision	recall	f1-score	support
0	0.91	0.98	0.94	250
1	0.25	0.07	0.11	27
accuracy			0.89	277
macro avg	0.58	0.53	0.53	277
weighted avg	0.84	0.89	0.86	277

Out of all the ML algorithms, Random forest and Gradient boost have a precision of 50% and accuracy of 90.25%. Since both the metrics are equal, we need to consider the next important metric that is recall.

Here recall value for random forest is 18.52 % and gradient boost is 14.81 %. For a good classifier, the recall value should be as high as possible.

Since recall value of random forest is higher, we can come to a conclusion that random forest algorithm is the best out of all the classification algorithms.